



University of Sharjah
College of Computing and Informatics
Department of Computer Engineering

Senior Design Project Report

Phase 2 – Senior Design Project 2

Detection of Diabetic Retinopathy using Artificial Intelligence

Project Group:

U19104151	Alia Albastaki
U19100721	Yosr Mohamed Elmudather
U19104506	Noora Alshehhi

Project Examination Committee

Supervisor	Dr. Ali Bou Nassif
Examiner	Dr. Talal Bonny
Examiner	Dr. Saddaf Rubab

Fall 2022/2023

Table of Contents

Table of Contents	ii
Abstract	iv
ملخص المشروع	v
Symbols, Keywords, and Abbreviations.....	vi
List of Tables	viii
Problem Statement	9
1 Introduction and Motivation	10
1.1 An Overview of Diabetic Retinopathy	11
1.2 Design Criteria	13
1.3 Our Design.....	13
1.4 Related Work	14
2 Alternative Solutions	21
2.1 IDx-DR system	21
2.2 Retinalyze	21
2.3 EyeArt.....	22
2.4 Bosch DR algorithm	22
3 Technical Background	23
3.1 Artificial intelligence	23
3.2 Machine learning	24
3.3 Deep Learning.....	24
3.4 Classification	26
2.4.1 ANN.....	26
2.4.2 Support Vector Machine	27
2.4.3 Random Forest	28
3.5 Convolutional neural networks.....	29
2.5.1 VGG architecture	30
2.5.2 DenseNet.....	31
2.5.3 MobileNetV2	32
4 Design Implementation	33

4.1	Proposed Design	33
4.2	Software Architecture	35
4.2.1	Programming Language	35
4.2.2	Integrated Development Environment	36
4.2.3	Image classification techniques	36
4.2.4	Convolutional Neural Network Architecture	37
4.2.5	Reset-50 Architecture.....	37
4.3.6	VGG-19 Architecture.....	38
5	Results and Discussions	40
5.1	Testing procedure	40
5.1.1	Datasets	40
5.1.2	Image Editing	41
5.1.3	AI Models.....	42
5.2	Experimental Results and Discussions	44
6	List of Constraints.....	50
6.1	Environmental Concerns.....	50
6.2	Ethical and Social Issues.....	50
6.3	Economic Issues	50
7	List of Specifications	51
7.1	Parts List and Estimated Budget.....	51
8	Difficulties Encountered	52
	Conclusion	53
9	References.....	54
	Appendices.....	59
	Appendix 1 – ResNet50 Python code.....	59

Abstract

The use of artificial intelligence (AI) in healthcare has shown improvement in diagnostic accuracy, especially in the area of diabetic retinopathy. Our project aimed to compare the performance of three convolutional neural network (CNN) models, ResNet50, VGG19, and CNN, to detect diabetic retinopathy using retinal fundus images. The dataset used in this project contained 3,662 retinal images. The models were trained using a transfer learning approach. This involves using a pre-trained convolutional neural network that has already been trained using large datasets. Our study found that ResNet50 had the highest accuracy, with an overall accuracy of 95%, followed by VGG19, 93%, and CNN, with an accuracy of 92%. The outcomes demonstrated the value of choosing the best CNN model for the task at hand and how AI can potentially improve DR diagnosis.

ملخص المشروع

ان الذكاء الاصطناعي يستخدم في الرعاية الصحية لتحسين دقة التشخيص، وخاصة في مجال مرض الشبكية السكري. ان هذا المشروع يهدف الى تطوير نظام الكشف المبكر القائم على الذكاء الاصطناعي والذي يستخدم صور الشبكية لتشخيص اعتلال الشبكية السكري. نظرًا لعدم وجود أعراض مبكرة، يمكن اكتشاف اعتلال الشبكية السكري باستخدام النظام القائم على الذكاء الاصطناعي في مراحله المبكرة، مما يسمح للمرضى باتخاذ الخطوات اللازمة لعلاج قبل أن يبدأ في إضعاف بصرهم. هدف هذا المشروع الى مقارنة أداء ثلاث نماذج شبكات عصبونية تابعة للتصنيف باستخدام صور أساس الشبكية. تحتوي المجموعه في الكشف عن مرض الشبكية السكري باستخدام ResNet50, VGG19, و CNN.

البيانات المستخدمة في الدراسة على تحتوي على ما يقارب ثلاثة آلاف وستمئة صوره لأساس الشبكية. تم تدريب هذه النماذج باستخدام نهج التعلم النقلي، حيث تم تعديل هذه النماذج مسبقا. حصد ResNet50 على اعلى نسبة، حيث بلغت نسبة الدقة على 95%، ثم أظهرت الدراسة ان VGG19 بلغ نسبة دقته 93%، يليهم في الأخير CNN حيث بلغت نسبة دقته 92%. أظهرت النتائج قيمة اختيار أفضل نموذج CNN للمهمة المطروحة وأظهرت كيف أن الذكاء الاصطناعي لديه القدرة على تحسين تشخيص مرض الشبكية السكري (DR).

Symbols, Keywords, and Abbreviations

AI	Artificial Intelligence
CL	Confidence Interval
CNN	Convolutional neural networks
DR	Diabetic Retinopathy
DL	Deep Learning
DM	Diabetes mellitus
DME	Diabetic Macular Edema
FAZ	Foveal Avascular Zone
GPIO	General-purpose input/output
HMA	Hemorrhage
ICDR	International Clinical Diabetic Retinopathy
KNN	K-Nearest-Neighbor
LCD	Liquid-crystal display
LED	Light-emitting diode
MA	Microaneurysm
ML	Machine Learning
MLP	Multi-level Perceptron
NPDR	Non-proliferative Diabetic Retinopathy
NRDME	Non referable DME
OCT	Optical Coherence Tomography
PDR	Proliferative Diabetic Retinopathy
SVM	Support Vector Machine
RF	Random Forest

List of Figures

Figure 1 Difference between Normal Retina and Diabetic Retinopathy [29].	11
Figure 2 Stages of DR [8].	12
Figure 3 The image processing steps (a) Source OCTA image (b) Structural image of microvascular with resolution enhancement (c) Binarized image with vessels reconstruction (d) A circle with 1.5 mm cantered at FAZ region (e) calculate the designed inter [9].	14
Figure 4 Example of the field of view examined by neural network[11].	16
Figure 5 Visual diagram of the DeepDR system[12].	17
Figure 6 Study procedures in the study of the CARE system[13].	18
Figure 7 Relationship between AI, ML and DL	23
Figure 8 The traditional structure of a neural network	24
Figure 9 A Neuron	25
Figure 10 Multilayer Perceptron diagram [29]	27
Figure 11 Convolutional Neural Network [22].	30
Figure 12 VGG-19 Layer Architecture [24].	31
Figure 13 DensNet architecture [26].	31
Figure 14 Block diagram.	33
Figure 15 Flowchart representation	34
Figure 16 ResNet50 Model Architecture	38
Figure 17 VGG19 Model Architecture	39
Figure 18 Graphical representation of scores on the APTOS Dataset	46
Figure 19. Training and Validation Accuracy Curve – ResNet50	47
Figure 20. Training and Validation Loss Curve - ResNet50	47
Figure 21. Confusion Matrix – ResNet50	48
Figure 22. Images from the dataset after distortion	48
Figure 23. Graphical representation of scores on the APTOS Dataset	49

List of Tables

Table 1 Related work limitations and results.....	19
Table 2. Experiments done in our project.	45
Table 3. Comparison between the models before and after modification	45
Table 4. Measures for VGG-19, ResNet-50 and CNN.	46

Problem Statement

In healthcare, diseases are effectively treated when diagnosed at an early stage. Diabetes is a disease that increases blood glucose due to a lack of insulin. It affects the kidneys, retina, heart, and nerves.

Diabetic Retinopathy (DR) is a diabetic condition that causes the retinal blood vessels to enlarge and leak fluids and blood. A new study predicts that by 2030, there will be 191.0 million people with DR, up from 126.6 million in 2010 [1]. It has no apparent symptoms during its initial stages until it gets more advanced. However, early signs of the condition can be detected using eye screening. These symptoms do not necessarily mean that the patient is diagnosed with DR. In its later stages, DR might result in vision loss. Patients that have developed diabetes are at a higher risk of developing diabetic retinopathy. Regular retina screening is crucial for diabetic patients to detect and treat DR early to prevent blindness.

Diverse types of lesions on a retinal scan indicate the existence of DR. Therefore, as engineers, we chose to explore the development of an AI that can detect retinal damage at an early stage. Our project aims to develop a high-performance AI as an alternative to the present screening for diabetic retinopathy that can be used at homes, pharmacies, and clinics with minimal effort and early diagnosis.

1 Introduction and Motivation

Diabetes mellitus (DM) is currently a global epidemic. By 2030, the number of people worldwide with diabetes mellitus (DM) is projected to reach 360 million, according to World Health Organization [2]. This epidemic has expanded to emerging nations due to changes in food habits, obesity, and physical inactivity. Diabetes mellitus DM is associated with microvascular consequences like diabetic nephropathy, neuropathy, and retinopathy. However, everyone with diabetes mellitus will be at risk for developing diabetic retinopathy (DR).

Diabetic Retinopathy DR is the most prevalent consequence of diabetes and is sometimes misdiagnosed until it has evolved to a vision-threatening stage. Reducing the incidence and progression of blindness requires earlier detection and prevention of DR progression. Non-proliferative diabetic retinopathy (NPDR) is the disease's early stage, while proliferative diabetic retinopathy (PDR) is the disease's late stage. To lessen the risk of severe vision loss, NPDR must be arrested early. However, present treatments target the late stages of DR, when vision has already been impaired; thus, it is necessary to prevent the progression of DR early. In addition, most advanced-stage treatments are costly, intrusive, and fraught with severe side effects.

For early detection of DR, ophthalmologists routinely suggest that diabetes patients undergo frequent fundus examinations. Nonetheless, diabetic retinopathies are typically overlooked until severe damage has occurred in a patient's fundus (usually manifested by loss of vision or deterioration).

Several developed nations have implemented a well-structured screening system to control the condition and deliver timely, high-quality care. The proper identification and categorization of DR assist clinicians in determining appropriate intervention measures, allowing prompt treatment administration. The high cost of operating these screening programs and the lack of qualified healthcare professionals have compelled the medical community to seek alternate methods for grading DR that save time and costs. Less developed countries also try to reduce the cost of DR grading. Using an automated computerized technique involving artificial intelligence (AI) is the current innovative method for resolving this problem [3]. Artificial Intelligence (AI) is the replication of human intelligence by a machine using complex algorithms that learn to discover patterns in data and then predict patterns in unseen data.

Our project's primary objective is to create a module with a system that allows for automated diabetic retinopathy DR detection and diagnosis. For evaluating the system, real-time retinal scans and parts of the datasets are integrated with deep learning and image processing. Image processing and artificial intelligence have proven their potential to effectively diagnose diabetic

retinopathy and enable retinal image analysis, which helps to influence the patient's treatment immediately.

1.1 An Overview of Diabetic Retinopathy

Diabetes is a chronic disease marked by an unusually high blood glucose level that destroys blood vessels. The high glucose level causes damage to the retina's nourishing blood vessels. Diabetic retinopathy (DR) is a condition where diabetes can harm the light-sensitive tissue and small blood vessels in the retina. By leaking the blood and fluid into the retina, these tiny blood vessels will cause microaneurysms, hemorrhages, hard exudates, cotton-wool patches, and venous[4]. Figure 1 shows the difference between diabetic retinopathy and normal retina.

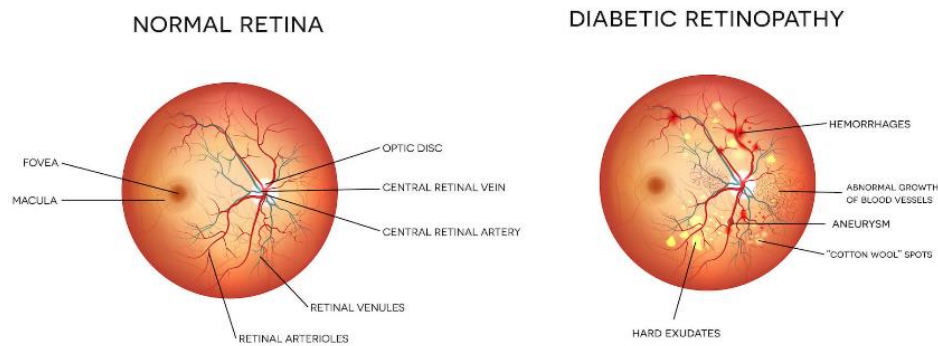


Figure 1 Difference between Normal Retina and Diabetic Retinopathy [29].

Physicians have outlined the stages of DR by identifying features and symptoms. A sample retinal picture with the common DR lesions that have been manually annotated is shown in figure 1.

The clinical features of DR are discussed below:

- **Microaneurysms:** Microaneurysms (MAs) are deformations in the vessel walls. These are recognized as balloon-shaped deformations caused by hyperglycemia-induced permeability of the vasculature. DR is directly proportional to the number of MAs.
- **Hemorrhages:** The formation of hemorrhages (HMAs) is caused by blood flow from damaged capillaries. Dot, flame, and blot are the three subcategories of HMAs.

The color of the dot HMAs is dark red. It is difficult to distinguish these from MAs based on color, but they are larger. Flame HMAs are distinguished by their flame shape. These are produced due to blood leaking along nerve fibers. Typically, blot HMAs are larger and have an irregular shape [5].

- **Hard Exudates:** Hard exudates appear on the retina as either brilliant yellow or white objects. Against the background of blood vessels, these items have a waxy look and sharp edges [6].
- **Soft Exudates:** Soft exudates or cotton wool spots due to arteriole blockage appear as fluffy white lesions with blurred and hazy edges. Exudates, including soft and hard exudates, are one of the most common early lesions of DR [5], [6].

1.1.1. Diabetic Retinopathy Stages Classification

The international clinical diabetic retinopathy (ICDR) scale is one of the most frequently used clinical scales and consists of five stages for DR: none, mild, moderate, severe, and proliferative. The ICDR classification is utilized by most artificial intelligence (AI) systems for the categorization and severity of DR [7]. The two levels of diabetic retinopathy are proliferative diabetic retinopathy (PDR) and non-proliferative diabetic retinopathy (NPDR). Mild, moderate, and severe non-proliferative diabetic retinopathy are subcategories of NPDR.



Figure 2 Stages of DR [8].

Proliferative diabetic retinopathy (PDR) encompasses changes that include the detection of enlarged blood vessels. Detecting HMA, MA, hard exudates, and soft exudates falls under Non-Proliferative Diabetic Retinopathy (NPDR).

Figure 2 shows the various stages of DR. In the early identification of DR, no alterations in the retinal images are noticed. However, in mild NPDR, microaneurysms are visible in one of the four quadrants. Images of the fundus reveal microaneurysms, hemorrhages, and exudates in moderate NPDR. Severe NPDR is characterized by the development of venous beading, microvascular abnormalities in the retina, and the formation of new blood vessels, known as neovascularization in PDR [8].

1.2 Design Criteria

To choose the finest design that perfectly addresses the main goal of our project, a list of criteria has been established. It includes reliability, usability, cost, and real-time.

- **Reliability:** The design will use artificial intelligence and deep learning approaches to analyze retinal images and detect diabetic retinopathy (DR) to achieve the highest precision and negligible error.
- **Usability:** The design will allow individuals to use the system without the assistance of an ophthalmologist.
- **Cost:** Our objective is to attain a price that is as competitive as possible with existing items.

1.3 Our Design

Convolution neural networks (CNNs) are a type of deep learning algorithm that uses image recognition tasks. To design a CNN to detect diabetic retinopathy, first step is to collect datasets of retinal images, both with and without signs of diabetic retinopathy. The next step is to train the CNN on the dataset chosen, using a technique known as supervised learning. During this process, the CNN learns to recognize the patterns and features in the images that are associated with diabetic retinopathy. The CNN is utilized to assess new retinal pictures and correctly identify the presence or absence of diabetic retinopathy after being trained. This technology has the potential to revolutionize the diagnosis and treatment of diabetic retinopathy, allowing for earlier detection and intervention, and ultimately saving vision and sight for many people.

1.4 Related Work

- Effective blood vessels reconstruction methodology for early detection and classification of diabetic retinopathy using OCTA images by artificial neural network

In this study [9], the diagnostic algorithms depend on retinal medical imaging methods, which can be classified according to invasive or non-invasive image techniques. Invasive techniques provide a 2D image of dynamic visualization of blood flow through retinal vessels. The non-invasive technique provides volumetric blood flow information images by the reflectance of laser light on the surface of moving red blood cells to depict vessels accurately through different segmented areas of the eye. Optical Coherence Tomography Angiography (OCTA) is the latest non-invasive technique. The image processing technique steps can be illustrated in Fig. 3

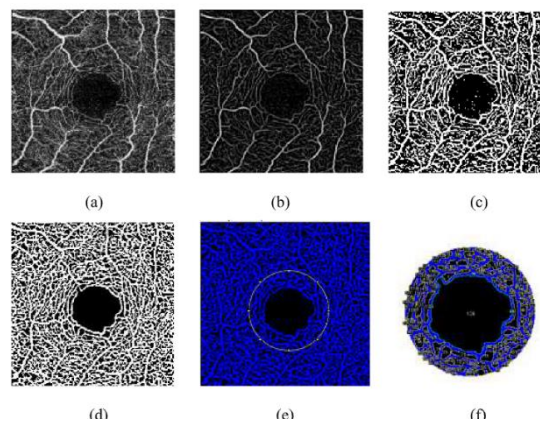


Figure 3 The image processing steps (a) Source OCTA image (b) Structural image of microvascular with resolution enhancement (c) Binarized image with vessels reconstruction (d) A circle with 1.5 mm centered at FAZ region (e) calculate the designed inter [9].

Diabetic retinopathy cases have large foveal avascular zone (FAZ) areas and perimeters compared to normal cases because of the beginning of vascular malformation. Artificial neural networks (ANN) are a set of algorithms modeled closely to the human brain; they are designed to perform almost nonlinear statistical modeling. It enables a computer to learn from observational data. It is an automatic classifier between the person with diabetes without DR and the Non-Proliferative diabetic retinopathy (NPDR). The objective of training the neural network is to minimize cumulative error. The quality of the studied images ranged between 40% and 65%; The proposed system is essential as it provides an accurate, automatic, and quantitative simple method of early detection. Using these features as inputs in ANN is helpful in the early diagnosis of DR. This methodology depends on using written custom programs

and a plugin for MATLAB and Fiji-based Image-J software with a supervised artificial neural network.

As a result, the total system accuracy reached 97%. The classification system performance parameters versus diabetic were 97.5% for sensitivity, 96.67% for specificity, and 95.2% for precision. In contrast, the measures for a person with diabetes without DR versus non-proliferative DR (mild to moderate) were 96.67% for sensitivity, 6.67% for specificity, and 96.67% for precision. The maximum misclassification error was 3.33%.

- Deep Learning Fundus Image Analysis for Diabetic Retinopathy and Macular Edema Grading

This study [10] was performed with anonymized retinal images of people from Finland and used the binary classification for DR; NRDR (nonRDR) and RDR and NRDME (non-referable DME) and referable DME. The accuracy was higher than in other similar studies, despite the training of the DL system being done with a lower number of images. They have shown that they achieved the best accuracy results while using the largest 2095×2095 pixels input image size, emphasizing that image quality is the key in AI-assisted DR diagnosis. It demonstrated that a deep learning AI system applied to a small retinal image dataset could accurately identify the severity of diabetic retinopathy and macular edema. Its accuracy was improved by using high-resolution and quality images. The algorithm achieved a sensitivity of 95%, specificity of 95%, and AUC of 95%. The model performance was evaluated at the operating point where the tuning set achieved 0.900 sensitivity.

- Automatic detection of diabetic retinopathy using an artificial neural network: a screening tool

In this study [11], a neural network was trained to recognize features of diabetic fundus images and evaluated for the sensitivity and specificity of recognition of these features. A back propagation neural network (Neural Works Professional II/Plus) was employed to analyze images of the fundi of 147 patients with diabetic retinopathy and 32 images from ordinary subjects by applying grids using a specially designed program in C language.

The network can be used to identify the presence of vessels, exudates, and hemorrhages with high predictive values. The neural network was able to detect patients with retinopathy with sensitivity and specificity values comparable to other screening methods such as optometrists, Polaroids from non-mydratic cameras, and diabetic physicians for which sensitivity values for the detection retinopathy have varied from 61% to 97% and specificities from 78% to

96%.6. For diagnosing diabetic retinopathy, the network attained a sensitivity of 88.4% and a specificity of 83.5%.

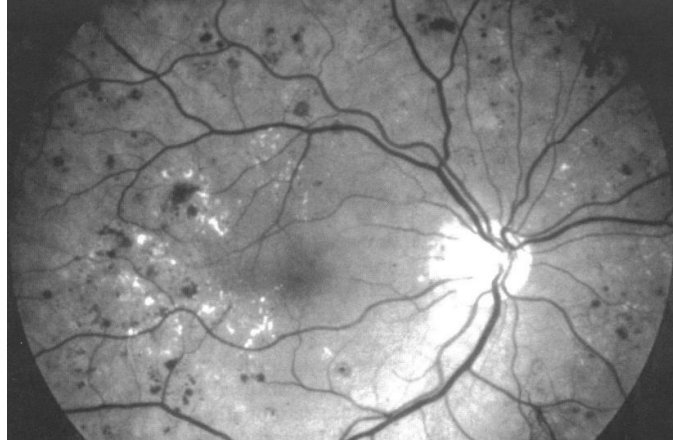


Figure 4 Example of the field of view examined by neural network[11].

- A deep learning system for detecting diabetic retinopathy (DeepDR) across the disease spectrum

DeepDR [12] was developed using the fundus images of patients with diabetes. 666,383 fundus images from 173,346 patients with diabetes with integrity fundus examination records were registered in this study. The prevalence of DR in the study in the training dataset, 12.85% of images had DR, among which 27.94% were mild NPDR. In the local validation dataset of 200,136 images, 12.99% of images had DR, among which 27.30% were mild NPDR. The local dataset was randomly divided into training or validation datasets.

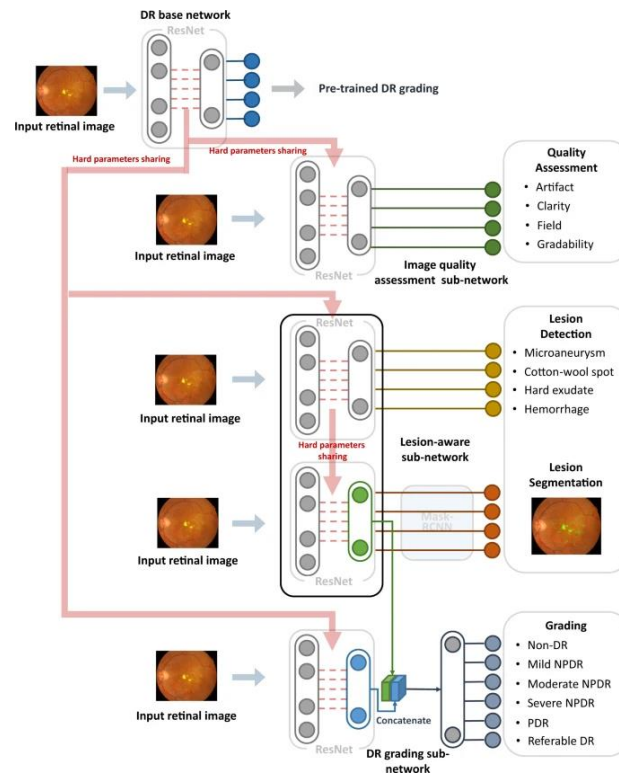


Figure 5 Visual diagram of the DeepDR system[12]

The DeepDR system had three sub-networks: the image quality assessment sub-network, the lesion-aware sub-network, and the DR grading sub-network. Python version 3.7.1 (Python Software Foundation, Delaware, USA) was used for all statistical analyses in this study. As a result, DeepDR enveloped an automated, interpretable, and validated system that performs real-time image quality feedback, retinal lesion detection, and early- to late-stage DR grading. The DeepDR system can improve image collection quality, provide a clinical reference, and facilitate DR screening with those functions. As a result, DeepDR achieved a sensitivity of 88.8% and specificity of 83.9% for mild NPDR detection on the local validation dataset.

- Evaluation of an Artificial Intelligence System for the Detection of Diabetic Retinopathy in Chinese Community Healthcare Centers

The study [13] used an artificial intelligence system to assess fundus photography of DR in a sample of 443 individuals (848 eyes) from Chinese community healthcare centers with a previous diabetes mellitus (including type 1 and type 2 diabetes mellitus) diagnosis. In this study, the sensitivity and specificity of AI for detecting any DR were 75.19% and 93.99%, respectively. As a result, (68.63%) had no DR, (3.89%) had mild NPDR, (14.27%) had moderated NPDR, 8.02% had severe NPDR, and 5.19% had PDR.

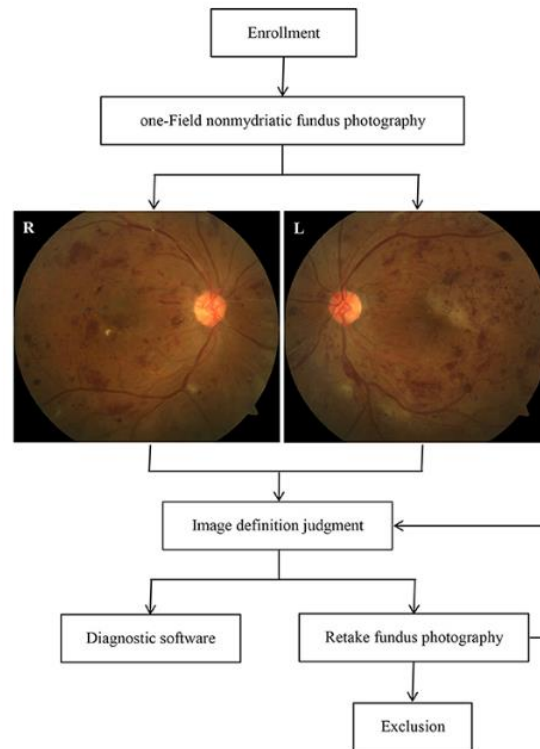


Figure 6 Study procedures in the study of the CARE system[13].

- Real-time diabetic retinopathy screening by deep learning in a multisite national screening program: a prospective intervention cohort study

As part of a national screening, this study [14] incorporated a deep-learning system into a routine screening method at primary care centers as part of the national screening. The deep-learning system also provided results in real time. The readings were accurate, a fundamental improvement for patients who had to wait weeks to months to get the results, and recommendations for referrals received them much sooner. In the vision-threatening diabetic retinopathy, the deep-learning system had an accuracy of 94.7%, 91.4% sensitivity, and 95.4% specificity.

- Performance and limitation of Machine Learning Algorithms for Diabetic Retinopathy Screening: Meta-analysis

This study Click or tap here to enter text. systematically examines the overall accuracy of the ML in diagnosing DR based on color fundus photographs and determining the state-of-the-art ML approach. Machine learning demonstrated high accuracy in diagnosing DR with various categories. The performance of ML in detecting more-than-mild DR was robust (sensitivity 0.95; AUROC 0.97). The neural network was the most widely used method, and the analysis revealed a pooled AUROC of 0.98 (95% 0.96-0.99) for studies that used neural networks to diagnose more-than-mild DR. A massive portion of the earlier published studies had methodology flaws, such as the lack of external validation and presence of spectrum bias. The results of these studies should be taken with caution.

Table 1 Related work limitations and results

Study	Limitations	Result
Effective blood vessels reconstruction methodology for early detection and classification of diabetic retinopathy using OCTA images by artificial neural network	This study aims to detect the initial stages, which may be confused based on the quality of the images or multiple diagnoses, to make the appropriate decision.	The total system accuracy reached 97%.
Deep Learning Fundus Image Analysis for Diabetic Retinopathy and Macular Edema Grading	The limitations in this study of deep learning AI-system consisted of image grading references where the two experienced and qualified experts may include grader biases that can decrease model performance. As well as, deep learning neural networks may inherit limitations of learning features that are unknown or ignored by medical experts.	The algorithm achieved a sensitivity of 95%, specificity of 95%, and AUC of 95%.

Automatic detection of diabetic retinopathy using an artificial neural network: a screening tool	The network performs poorly when evaluated with data it has yet to see because of a lack of generalization.	The network achieved a sensitivity of 88.4% and a specificity of 83.5%.
A deep learning system (DeepDR) for detecting diabetic retinopathy across the disease spectrum	Due to the lack of lesion annotations in external cohorts, the lesion-aware sub-network was evaluated only on the local validation dataset.	As a result, DeepDR achieved a sensitivity of 88.8% and specificity of 83.9% for mild NPDR detection on the local validation dataset.
Evaluation of an Artificial Intelligence System for the Detection of Diabetic Retinopathy in Chinese Community Healthcare Centers	Due to the lack of ophthalmic expertise of primary physicians, DR diagnosis and referral could be more timely.	The sensitivity and specificity of AI for detecting any DR were 75.19% and 93.99%, respectively.
Real-time diabetic retinopathy screening by deep learning in a multisite national screening program: a prospective intervention cohort study	In this study, they developed their costume tracking process because electronic medical records and referral tracking were unavailable within the screening program.	The deep-learning system had an accuracy of 94.7%, 91.4% sensitivity, and 95.4% specificity.

2 Alternative Solutions

Many techniques are used in healthcare systems to detect DR, such as digital fundus imaging. Although, access to fundus photography is often limited by patient morbidity, high equipment cost and a shortage of trained personnel. Conventional imaging approaches also require patient cooperation with stabilized, upright head positioning, which can be difficult among sick, wheelchair-bound, and immobilized patients and children [16]. Hence, many solutions have been made in previous years to detect DR by engineers, Diabetologists, Ophthalmologists and researchers. Deep learning and artificial intelligence advancements have improved the probability of widespread automated screening, which can successfully detect diabetes at an early stage [17].

2.1 IDx-DR system

IDx-DR is an autonomous AI system for the real-time, point-of-care diagnosis of diabetic retinopathy. The operator assistant uses a fundus camera to take high-quality images of the patient's eyes. The images are then submitted to IDx-DR, where they are analyzed for signs of diabetic retinopathy (including diabetic macular edema). The pictures are then analyzed and labeled by a web-based server called Retinalyze. Images are labeled to determine whether retinal abnormalities have been detected. It also provides an illustrated version of the image to highlight suspect DR changes. Results are per image and each image is screened individually.

While IDx-DR is dependable, it could be better. IDx-DR may be unable to detect present disease or give a false positive diagnosis [18]. A problem with this system is that it requires operator assistance to achieve the reported diagnostic accuracy and diagnosability used to acquire all images in this study.

2.2 Retinalyze

The Retinalyze software used a supervised learning model from already human-labeled training data (fundus photos) to train to recognize diabetic retinopathy and age-related macular degeneration lesions in the retina. It is a web-based service that analyses each image separately and labels each image regarding whether retinal abnormalities have been detected. It also provides an annotated version of the image with the suspected DR changes highlighted. The results are per-image, and each image is screened separately. A disadvantage of Retinalyze is that it can be accessed online or integrated into a healthcare center's existing software systems. This technology needs someone to check the results and claims detection of diabetic retinopathy but not the diagnosis [18]. For results due to vision

express Poland, it showed the sensitivity and specificity of RetinaLyze software to detect DR as high as 97.0% and 96.7%.

2.3 EyeArt

"EyeArt system utilizes novel image analysis techniques customized for DR screening and is engineered for large scale deployment on the cloud" [19]. The EyeArt system is a computerized, cost-effective, and cloud-based AI medical device. Endocrinologists, diabetologists, and general practitioners can easily use EyeArt to rapidly and accurately screen color fundus images and generate a report within minutes. EyeArt is designed to work with most types of retinal cameras. The algorithm used in EyeArt has been trained using data from over half a million people and two million retinal images. Color fundus images are uploaded to the technology's General Data Protection Regulation (GDPR) compliant cloud. The technology grades the severity of diabetic retinopathy and reports results for each patient's eye. EyeArt screening had 91.3% sensitivity and 91.1%.

2.4 Bosch DR algorithm

The Bosch DR Algorithm uses a deep convolutional neural network to detect whether a fundus image has DR automatically. The recently described automatic DR screening solution from Bosch employs a Bosch Mobile Eye Care fundus camera. The images taken by the fundus camera are analyzed by a convolutional neural network-based AI software to deliver a disease/no-disease or poor-quality output [20]. The Bosch DR Algorithm achieved impressive results with sensitivity and specificity rates of 91% and 96%.

3 Technical Background

This chapter provides a concise outline of the necessary background ideas important in understanding detecting DR. In this chapter (ML), the idea of artificial intelligence (AI) and the core ideas of machine learning are highlighted, the basis of deep learning, and its connection to machine learning. In addition, the different techniques we will use to extract, pre-process, and classify will be explained.

3.1 Artificial intelligence

Artificial intelligence (AI) emulates human intelligence in systems designed to behave and think like humans. It can also be referred to as any computer that demonstrates characteristics of the human intellect, like learning and problem-solving. Since its establishment as a field of study in 1955, artificial intelligence has undergone several waves of optimism, followed by failures known as the 'AI winter,' then new strategies, achievements, and increased funding [21]. Knowledge representation, reasoning, learning, planning, natural language processing, speech processing, and many more subfields in AI are all included. Automated reasoning is an area that involves knowledge representation and reasoning to help computers to understand and apply various aspects of reasoning. Natural language processing allows the machine to understand human language, and these developed computers can process enormous amounts of natural language and analyze it. Machine perception is the capability of a computer system to interpret data like how humans' sense, feel and see the things around them, which falls into the category of machine learning. Hence, machine learning (ML) is a subpart of AI.

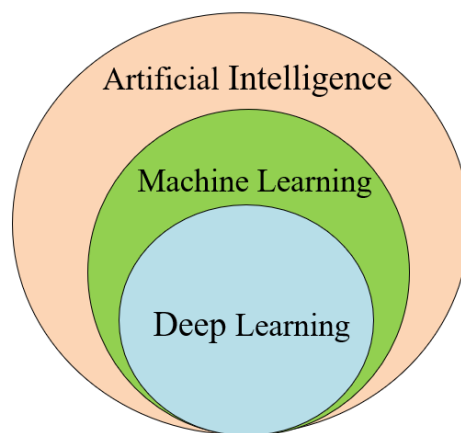


Figure 7 Relationship between AI, ML and DL

3.2 Machine learning

Machine learning is a subset of AI which allows the machine to learn from data implicitly without being programmed. Machine learning types have traditionally been divided into three categories corresponding to learning paradigms, depending on the type of "signal" or "feedback" available to the learning system. These types are supervised learning, unsupervised learning, and reinforcement learning. Supervised learning deals with two primary tasks regression and classification. The AI model is trained to provide a result for a given input based on the mapping equation created for the input. In unsupervised learning, the AI model does only one task: classification. The model classifies the input data into sub-parts with similar features and then labels the input after prediction based on similarities and patterns found. The reinforcement learning model learns by getting feedback from past outcomes [22].

3.3 Deep Learning

Deep learning is a subset of Machine Learning. It uses numerous layers to extract higher-level features from the raw input [23]. The biological neural networks of the brain inspire it. All deep learning methods use neural network architectures. Traditional neural networks contain only 2-3 hidden layers: the input layer, hidden layers, and output layer. The input layer takes the input from the data. It passes it to the hidden layers that perform a nonlinear combination of the information from the previous (input or hidden) layers [24]. All these layers are composed of a neuron-like structure of the biological neuron of the human brain. The building block of a traditional neural network is shown in figure 8.

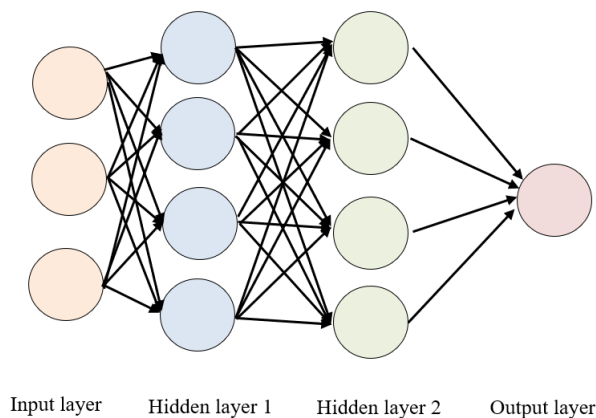


Figure 8 The traditional structure of a neural network

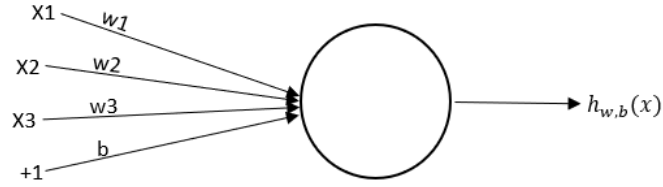


Figure 9 A Neuron

Figure 9 shows the simple mathematical model of a single neuron as proposed by McCulloch and Pitts in the 1940s [25]. A simple linear model produced a positive or negative output, given a set of inputs and weights. This model of computation was intentionally called neuron because it tried to mimic how the core building block of the brain worked. Just like brain neurons receive electrical signals, McCulloch and Pitts' neurons receive inputs and pass them on to other neurons if these signals are strong enough. As shown in figure 8, The input layer of a conventional neural network receives an input and sends it on to the hidden layers, which produce an output depending on the activation functions. A unit (individual neuron) has parameters which are weights w (a $[n \times 1]$ vector) and a bias scalar which is denoted by b . An activation (a) function is represented by:

$$a = \sum_{i=1}^n x_i * w_i + b$$

A point-wise non-linear function denoted by $\sigma(\cdot)$ is applied to generate the output

$y = f(a) = \sigma(\sum_{i=1}^n x_i w_i + b)$. One use of the non-linear function σ is the rectified linear unit (ReLU). Other non-linear variants are Tanh, Sigmoid and leaky ReLU.

Once the architecture has been selected regarding the number of units/layers, the next step is to train the chosen model. The following are steps to set parameters.

1. Given dataset $\{x, y\}$, pick an appropriate cost function denoted by C .
2. Forward pass the examples through the model to get predictions.
3. Calculate the error using cost function C to compare the predictions.
4. Use back-propagation to undergo the error through the model, adjusting the parameters to minimize loss or energy E .
5. Once the gradients are obtained, use Stochastic Gradient Descent (SGD) to update the network's weights.

In SDG, we start with some initial set of parameters θ^0 , and the updates of the parameters are denoted by $\theta^{k+1} \leftarrow -\theta^k + n\Delta\theta$, where k is the iteration index and n is the learning rate, which is scalar. The gradients are computed by back-propagation, given $\Delta\theta = \partial E / \partial \theta$.

The back-propagation algorithm involves the following:

1. A forward pass model with N layers, with layer i having a vector of weight W_i . Each layer receives the input, and the output of each layer is computed from the top layer, $x_i = F_i(x_{i-1}, W_i)$.
2. A backward pass computes the cost derivative iteratively from the top layers to the bottom layer. $\frac{\partial C}{\partial x_{i-1}} = \frac{\partial C}{\partial x_i} * \frac{\partial F_i(x_{i-1}, W_i)}{\partial x_{i-1}}$.
3. Compute the gradients and update the weights to find the optimal weight.

SGD has a fixed learning rate (η). Learning rates dramatically affect the resulting model, and hence, having a constant learning rate is not optimal. Techniques such as AdaGrad, RMSProp, ADAM, and Batch-size are used to optimize the learning rate. Momentum is used to help speed up convergence, and the updated weight becomes $\theta^{k+1} \leftarrow \theta^k + \alpha \Delta\theta^{k-1} - n\Delta\theta$ after adding momentum, with α usually around 0.9 [26].

3.4 Classification

Many approaches are used in the classification, such as ANN, SVM, KNN, random forest, and CNN models, such as DenseNet121, VGG19, and MobileNetV2.

2.4.1 ANN

Artificial neural Networks (ANN) are a subset of machine learning. A neural network takes in data, uses learning algorithms to train and recognize patterns, and predicts output to be applied to future data.

- Multilayer Perceptron

Multilayer Perceptron is a neural network where the mapping between inputs and output is nonlinear. A multilayer perceptron has input, output, and one or more hidden layers with densely packed layers of neurons. In contrast to the Perceptron, where each neuron's activation function must impose a threshold, neurons in a Multilayer Perceptron are free to choose any activation function they like. The inputs are combined with the initial weights in a weighted sum and subjected to the activation function, just like in the Perceptron. However, the difference is that each linear combination is propagated to the next layer. The result of the

activation function determines whether a particular neuron will be activated. An activated neuron transmits data to the neurons of the next layer over the channels.

Hence, the data is transmitted through all layers, and this process is known as forwarding propagation. After getting predictions from the output layer, the actual output is compared with the predicted output, and the prediction error is calculated [27]. The error size indicates how wrong the output is. The input and output pairs are then presented to the network to reduce the error between the predicted output and the actual output by adjusting the weights. This process is known as backpropagation. Forward propagation and backpropagation are performed iteratively. After each forward pass, backpropagation performs a backward pass while adjusting the weights and biases. This process continues until the network can produce the correct output [28].

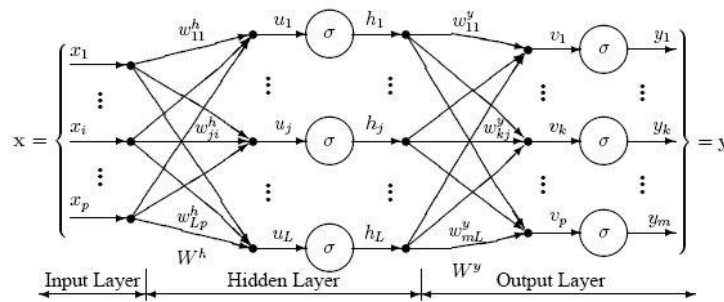


Figure 10 Multilayer Perceptron diagram [29]

2.4.2 Support Vector Machine

SVMs have supervised machine learning models that use learning algorithms to find a hyperplane and classify and separate all training vectors into two data classes. To separate the hyperplane into two vectors, many hyperplanes can be chosen. Although, SVM looks for a plan that has the most significant margin, having a greater distance between data points, to ensure that future data points will be classified most optimally [30]

- Linear SVM

Linear SVM is used for data that can be separated linearly. A hyperplane is drawn to classify the vector into two classes distinctly. The distance between the support vectors and the hyperplane line is called the margin. The margins width (w) needs to be maximized to select an optimal hyperplane, and the chosen labels need to be the farthest from the data points [31].

A hyperplane is expressed as:

$$w^{\rightarrow} x + b = 0$$

Where w is a vector normal to the hyperplane and b is an offset representing the intercept. If the equation results in a value greater than one, then the input values belong to the first class. Otherwise, the input vectors will be in the second class.

$$\begin{aligned} w^{\rightarrow} x + b &\geq 0 \text{ for } y_i = +1 \\ w^{\rightarrow} x + b &< 0 \text{ for } y_i = -1 \end{aligned}$$

given the following training point datasets:

$$\{x_i, y_i\} \text{ where } i=1,2,3\dots k \text{ and } y_i \in \{-1,1\}$$

- Nonlinear Kernels

Nonlinear SVMs are achieved by applying a kernel trick to a linear SVM. Kernel functions transform nonlinear spaces into linear spaces by transforming data into another higher dimension to classify the data. In the higher dimensional space, the input vectors are mapped, which makes the nonlinear models into linear models, where the classes are separable [32].

SVM can be used for data with irregular and unknown distributions and can also manage high complexity problems. Its execution time is faster and has better accuracy than the ANN algorithm [33].

2.4.3 Random Forest

A supervised learning algorithm called random forest is constructed from several decision trees. It is made with various decision trees on different samples, which take their majority vote for classification. Random forest is an extension of a technique called bagging.

- Bagging

Bagging is a technique that chooses a random sample from a data set, resulting in different trees. The final output is based on the majority voting after combining all trees [34].

One of the significant advantages of Random Forest is that it reduces overfitting, even if more trees are appended to the forest. It can also manage data sets containing continuous variables.

A significant drawback of random forest is that it requires time for training since many combinations of decision trees are made to determine a class [35].

3.5 Convolutional neural networks

Deep neural networks include convolutional neural networks (CNN). For object or picture recognition and classification, CNN is frequently used. Each neuron in one layer connects to every neuron in the following layer, forming a complete connection, and this is viewed as a multilayer perceptron. Additionally, compared to other image classification algorithms, it requires little pre-processing [36]. The CNN is a feed-forward model, where the input is fed to the convolution layers, non-linearities, pooling layers and finally to feature maps. As shown in figure 11, the convolution and pooling layers constantly alternate to reduce the computation time and to understand spatial details. The convolutional layer is the first to extract various features from the input image. This layer performs the mathematical operation of convolution between the input image and a filter of a specific size $M \times M$. Sliding the filter over the input image forms the inner product between the filter and the portion of the input image with respect to the size of the filter ($M \times M$). The filter is learned during the training process. Pooling is done to reduce the spatial size of the representation, thus reducing the number of parameters and preventing overfitting [37].

ImageNet is a large-scale image database with 14 million labelled images and 20 thousand classes, having pictures gathered from the internet. The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) is an annual software contest where software programs compete to build high-performing networks. In 2012, a CNN named AlexNet achieved a top-1 and top-5 error rates of 39.7% and 15.3% in the ImageNet 2012 Challenge, which was better than the previous results [24].

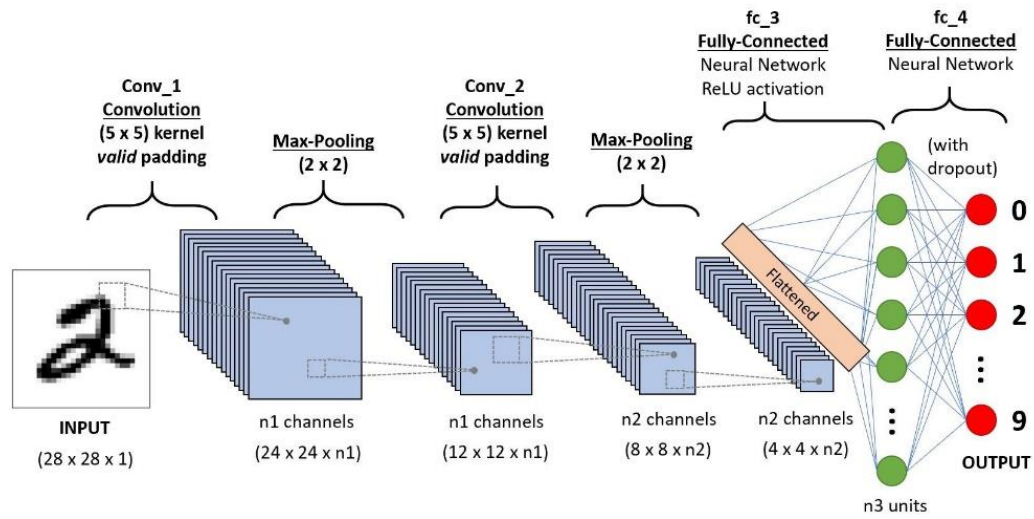


Figure 11 Convolutional Neural Network [22]

2.5.1 VGG architecture

VGGNet stands for visual geometry group network and multi-layered deep neural network. It consists of groups of blocks, where each block is composed of convolution and pooling layers. VGG comes in 2 models, VGG16 and VGG19, with 16 and 19 layers. The VGG is a CNN-based network applied to the ImageNet dataset, containing 1.2 million general object images of 1,000 different object categories for training. The primary purpose of the VGG is to show how classification can be improved by increasing the depth of the CNN. According to K. Simonyan and A. Zisserman from the University of Oxford in the paper [24] proposed in 2014, the VGG-19 CNN architecture is reported to achieve high accuracies for image processing large datasets such as ImageNet. VGG-19 model has 143 million parameters learned from the ImageNet dataset. In Figure 12, we presented that VGG 19 is a simple network and is

advantageous due to its simple nature. The convolutional layers are placed on the top to extend the depth level, and max-pooling stages are employed to minimize volume size.

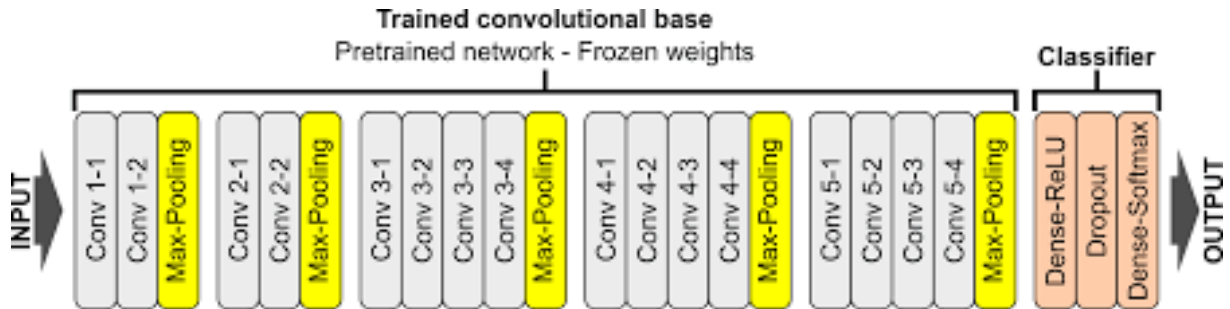


Figure 12 VGG-19 Layer Architecture [24].

2.5.2 DenseNet

DenseNet is a type of dense convolution neural network. DenseNet is a convolutional neural organization where each layer relates to any non-subsequent layers. To maintain the feed-forward nature, each layer obtains additional inputs from all previous layers and passes on its features to all subsequent layers. DenseNet comprises two significant units other than the convolutional and pooling layers: Dense blocks and transition layers. [25].

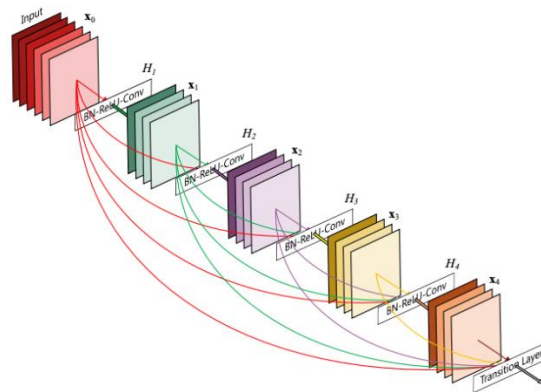


Figure 13 DensNet architecture [26]

2.5.3 MobileNetV2

MobileNetV2 is a lightweight model with fewer parameters in the deep neural network. The MobileNetV2 models are faster due to the reduced model size and complexity. Hence, MobileNetV2 is suitable for embedded systems. MobileNet architecture is based on depth-wise separable convolution. The depth-wise convolution splits the image and the filter into two channels and then convolves each input with the equivalent filter channel. The filtered values are then combined to create new features. The depth-wise separable convolution block performs the same output as the standard CNN but is more efficient due to the reduced size of parameters [27].

4 Design Implementation

In this chapter, we will describe the overall design of our project. The primary step that will be conducted in our project is acquiring retinal images. These pictures are then exported to the software using Python code to perform steps such as image pre-processing, feature extraction, and classification.

4.1 Proposed Design

The block diagram in Figure 14 shows the steps performed throughout our project. Our project consists of two main parts: training and testing. First, images would be acquired from the APTOS2019 dataset. Then the images would undergo many techniques, such as image pre-processing, data acquisition, data augmentation, and feature extraction. These images would then go through three models: CNN, VGG19, and ResNet-50. Finally, the model will be tested and trained.

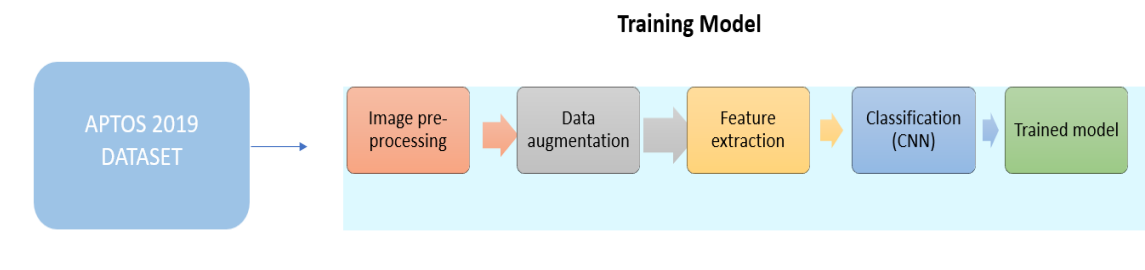


Figure 14 Block diagram

- Flowchart

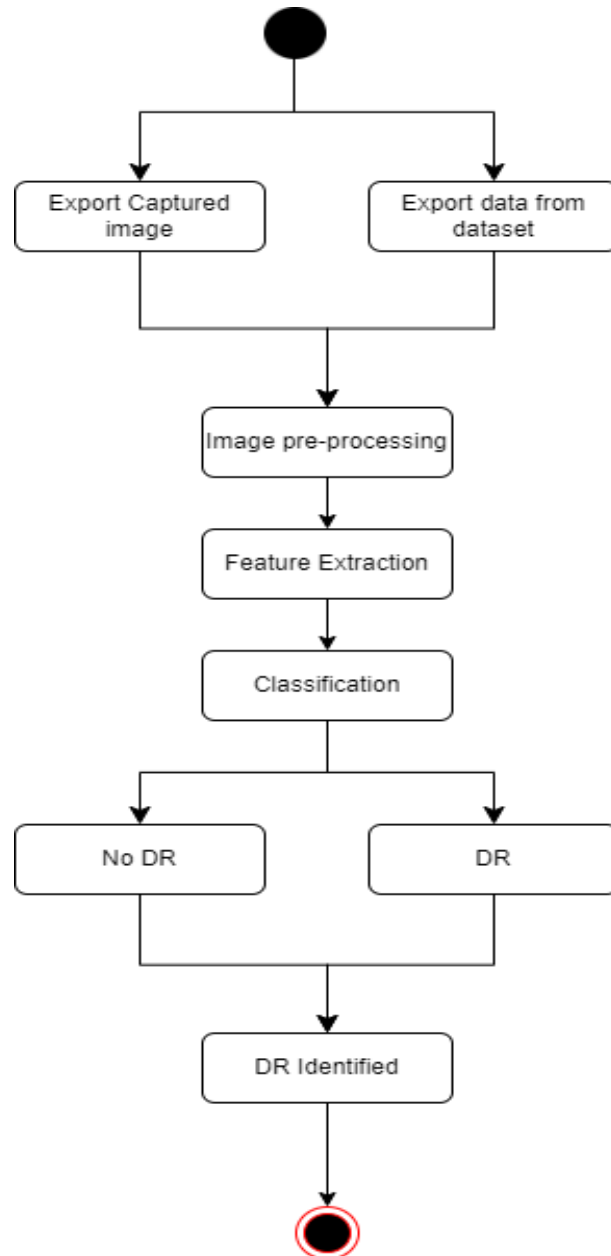


Figure 15 Flowchart representation

The flowchart in Figure 15 represents the software of our system. Once the images are pre-processed and extracted, they will be classified. The output of the system depends on the various stages of diabetic retinopathy. The five stages of diabetic retinopathy are no DR, mild DR, moderate DR, severe DR, and proliferative DR.

4.2 Software Architecture

In this chapter, we will lay the foundation elements of the research by explaining the datasets used, the pre-processing techniques, and performance evaluations. A successful CNN that detects diabetic retinopathy will depend on the data quality, architecture design, optimization, and hyperparameters. It is important to carefully design and evaluate the model to ensure that it is accurate and clinically useful. CNN is made up of several major components. Each component is essential in the network's ability to learn and classify images. Convolution layers, pooling layers, activation functions, fully connected layers, loss functions, and optimization methods are some of these components. These tools are powerful to recognize images, detect objects, and diagnose medical.

These components include:

- Convolution layers learn features from retinal images relevant to detecting diabetic retinopathy.
- Pooling layers to down-sample the feature maps outputted by the convolution layers, reducing the computational cost of the network. It will be able to learn more abstract representations of the image features.
- Activation functions enable the network to model the non-linear relationship between the input image features and the output classification. An activation map's negative values are removed by the ReLU layer by setting them to zero and applying an activation function to the feature map.
- Fully connected layers: to process the learned features from the convolution and pooling layers and output a final classification decision. It will indicate whether the input image contains signs of diabetic retinopathy or not.
- Loss function: During training, the loss function is critical to ensure that the network learns to classify diabetic retinopathy accurately. The binary cross-entropy will measure the difference between the predicted and actual classification in the case of diabetic retinopathy.
- Optimization algorithm: The optimization algorithm used to train the CNN is essential to ensure that the network converges to a good solution, minimizing the loss function over the training data and generalizing well to unseen data.

In summary, combining all components will allow CNN to learn mapping between retinal images and their corresponding classifications, making it a powerful tool for detecting diabetic retinopathy.

4.2.1 Programming Language

The identification of diabetic retinopathy using artificial intelligence and Python is a cutting-edge use of machine learning in healthcare. Python's portability and user-friendliness make it effective for this application. Python's flexibility to operate on a variety of platforms and

operating systems makes it an ideal choice for creating a portable diabetic retinopathy detection system. Python's high-level syntax and extensive libraries make artificial intelligence models for diabetic retinopathy detection simple to develop and deploy. The language's strong support for machine learning frameworks such as Keras allows it to provide a variety of pre-built models and tools for image recognition and classification applications.

To develop our diabetic retinopathy detection system, we used an integrated development environment (IDE) specialized for Python. This enables us to create, test, and debug our code all in one environment, speeding the development process and increasing productivity.

4.2.2 Integrated Development Environment

Integrated development environment (IDE) software provides a wide range of tools for developing and managing software. Codes can be written, tested, and debugged by programmers. A Google Collab online program offers a Jupyter Notebook environment where Python programs may be written and executed. It is also suitable for experimenting with machine learning models. An advantage of Google Collab is that it is a free and cloud-based service that allows running Python code on Google servers without installing any software. It also allows free access to GPU resources, which helps in reducing the time needed to train machine-learning models. We were capable of finding bugs or errors in the early phases of development due to the user's ability to run each cell of code.

4.2.3 Image classification techniques

Common classification techniques used in CNNs to detect diabetic retinopathy are:

- **Pre-processing:** First, we must pre-process the images to reduce noise and enhance features. It may involve resizing the images, normalizing the pixel values, and applying filters to enhance edges or remove noises.
- **Data augmentation:** The CNNs can be trained using data augmentation techniques to create new training examples of the original images. Images can be rotated, flipped, or cropped to create a variation of the original images.
- **Transfer Learning:** The transfer learning involves using a pre-trained CNN that has been already trained using large datasets, such as ImageNet. This is useful when there is limited training data available.
- **Feature extraction:** CNNs consist of multiple layers to extract features from the input images. In the case of diabetic retinopathy, these features may include tiny red dots (microaneurysms) and bright red patches on the white of the eye (hemorrhages).
- **Classification:** Finally, a CNN typically performs classifications using the extracted features. In diabetic retinopathy, these classifications include whether DR is present or not.

4.2.4 Convolutional Neural Network Architecture

Convolutional neural networks (CNNs), also known as deep neural networks, are widely utilized in image recognition and computer vision applications. Consequently, their ability to automatically extract and learn characteristics from pictures, they are very good at tasks like object identification, image categorization, and segmentation.

A typical CNN architecture comprises input, convolutional, RELU, pooling, and fully connected layers. The first layer receives and transmits the input picture to the following layer. The convolutional layer uses the input picture to apply a series of filters, creating a series of feature maps that emphasize various facets of the image. The rectified linear unit (ReLU) activation function is used for the feature maps in the ReLU layer, which gives the Network non-linearity and enables it to learn more intricate features. The pooling layer downsamples the feature maps and lightens the network's computational load by applying a pooling function—such as max pooling or average pooling—which reduces the size of the feature maps. Moreover, the fully connected layer adds a set of weights to the preceding layer's flattened output to create the final output.

There are several variants of CNNs, each with distinct architecture and use. Thus, we will discuss and employ three different architectures, Resnet50, VGG19, and a standard CNN model architecture.

4.2.5 Reset-50 Architecture

Resnet-50 is a convolutional neural network architecture for image classification tasks. The Resnet (short for Residual network) consists of 50 layers, including convolutional, pooling, and fully connected layers. By bypassing layers, the architecture employs residual connections to let information flow directly through the network. This makes building far deeper networks and helping with the vanishing gradient problem possible.

The architecture is composed of several blocks, each containing multiple convolution layers, batch normalization layers, and activation functions. These blocks are connected in a sequence, with each block's output feeding the one before. The final output is run through a fully connected layer with a softmax activation function to create a probability distribution over the potential classes.

The equation of a single ResNet block can be expressed as follows:

$$H(x) = F(x) + x$$

Where 'x' is the input to the block, 'F(x)' is the output of the convolution layers of the block, and 'H(x)' is the block's final output after the residual connection's addition. The vanishing

gradient problem can be avoided, and the performance of intense networks can be enhanced by allowing the gradient to pass directly through the block without passing through the convolutional layers.

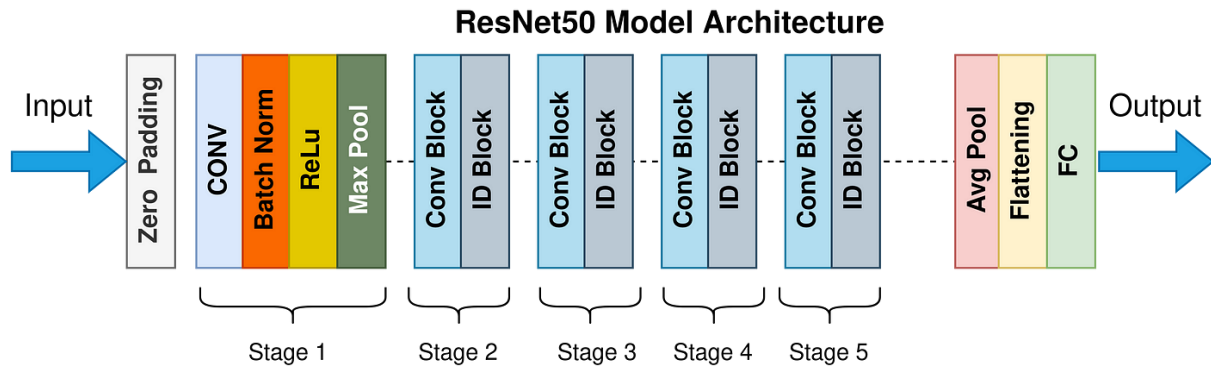


Figure 16 ResNet50 Model Architecture

On several image classification tasks, including the ImageNet dataset, which comprises over a million photos divided into 1,000 classes, ResNet50 has demonstrated state-of-the-art performance. Due to its success, more complex ResNet architectures, such as ResNet101 and ResNet152, have been created.

4.3.6 VGG-19 Architecture

VGG19 is a deep convolutional neural network (CNN) architecture with 19 layers, including 16 convolutional layers, three fully connected layers, and five pooling layers. The layers are arranged in sequential order, and the input to the network is an RGB image of 224x224 pixels. It is widely used for image classification and other computer vision tasks.

The VGG19 architecture uses small 3x3 filters with a stride of 1 pixel, which allows the network to learn more complex features from the input images. The first layers in the network learn simple features, such as edges and corners, while the deeper layers learn more complex features, such as object parts and textures. The pooling layers in VGG19 use max-pooling with a 2x2 filter and a stride of 2 pixels, which reduces the size of the feature maps and helps to prevent overfitting. The fully connected layers in VGG19 have 4096 units each and are followed by a final softmax layer that produces the output probabilities for each class.

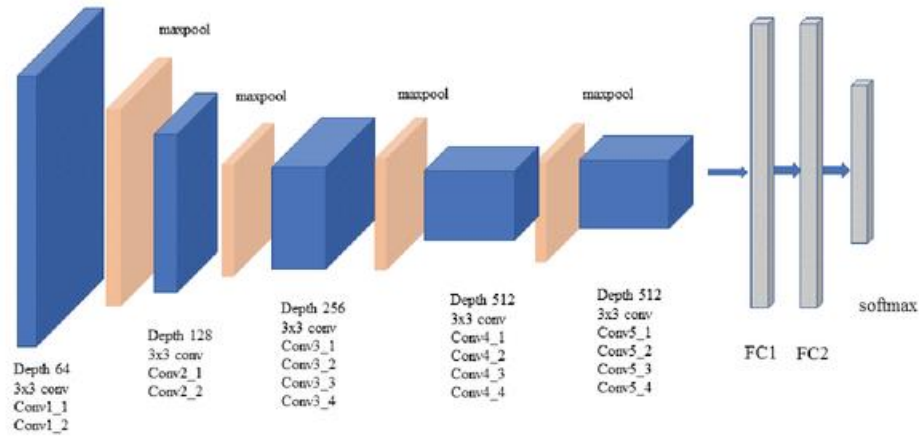


Figure 17 VGG19 Model Architecture

In the context of diabetic retinopathy detection, To categorize retinal pictures as healthy or diseased, VGG19 can be utilized as a trained model. A massive dataset of retinal pictures may be used to fine-tune the pre-trained model to learn to accurately recognize diabetic retinopathy indicators, which can aid medical professionals in diagnosing and treating the condition more successfully.

The model uses transfer learning by loading a pre-trained VGG19 model that has been trained on ImageNet, a large dataset of images. The final layer of the pre-trained model is removed, and a new output layer is added to classify the images as either having diabetic retinopathy or not. A global average pooling layer, a completely connected layer with 256 neurons and a ReLU activation function, and a final completely connected layer with two neurons and a sigmoid activation function are included in the new layers.

5 Results and Discussions

5.1 Testing procedure

In our study, an extensive collection of retinal pictures from individuals with and without diabetic retinopathy will be used to train the AI system. The photos would be labeled to show if diabetic retinopathy was present or not, as well as how severe the condition is.

We built and trained three different algorithms to test the dataset of retinal images, CNN, ResNet-50, and VGG19.

In the following sections, we will discuss our work from the second part of the senior design project.

5.1.1 Datasets

Many datasets were used to train models using pre-trained networks. i.e., the APTOS 2019 Blindness Detection dataset, a publicly available Diabetic retinopathy dataset from Kaggle.

The dataset contains retina fundus images to classify various levels of diabetic retinopathy. The fundus photos for dataset development are obtained from a local hospital and are captured with a digital fundus camera. Both left and right eye samples of the patients are provided. The ophthalmologist scaled each sample from 0 to 4 for each DR class. A level of 0 indicates a healthy case, while ratings 1, 2, 3, and 4 show mild, moderate, severe, and proliferative DR, respectively. These scales are employed as labels in our proposed model.

The proposed approach differs from the existing system, which distributes fundus images based on the clinical and pathological changes in the retina. Pictures are divided and placed in different folders according to their tags. The process of dividing the images into distinct categories is done under the guidelines of an expert ophthalmologist. After that, the images are cropped, and the key features are separated. A filtering technique is also used to equalize and contrast the image modification. The third step is to do data augmentation. Data augmentation is a technique for increasing the range of data available that has been adopted. After that, flipping, cropping, rotating, zooming, and padding are done.

Further, we gathered two publicly available datasets to train and test our models to build an accurate and efficient system.

- **EyePacs dataset:** The EyePACS dataset is a publicly available dataset from the United States. It has provided a large dataset of retinal images from diabetic screening programs. The images provided by EyePACS were captured under various conditions by various devices at multiple primary care sites throughout California and elsewhere. It consists of 88,702 fundus images, including 35,126 (35.6%) samples for training,

3,511 (4.0%) for validation, and 53,576 (60.4%) samples for testing. The competition provided class labels for the training, validation, and test images. Two images of the left and right eyes were collected for each subject with the exact resolution. A clinician was asked to rate each image for the presence of DR with a scale of 0–4 according to the severity scale. A certified reader rated all images according to a standard diabetic retinopathy grading scale[43]

- **APTOS 2019 dataset:** APTOS stands for Asia Pacific Tele-Ophthalmology Society. It is a subset of the EyePacs dataset obtained after performing some 33 preliminary operations. It consists of 3000+ images and is available in different formats [44].

The APTOS 2019 dataset has a large and diversified size and includes a wide range of disease severity and image quality, which make it an ideal dataset for the diagnosis of diabetic retinopathy. Ophthalmologists assess the dataset, giving AI algorithms a trustworthy source of truth. Additionally, the dataset is widely accessible to researchers and has a standardized grading system that makes it simpler to compare findings across studies and datasets. Overall, the APTOS 2019 dataset is the best option for diabetic retinopathy identification using AI due to its size, diversity, expert grading, standardization, and accessibility.

We performed data normalization on images from the datasets so that each test and train file had the same number of images. Data normalization was done to ensure that the dataset is not biased towards the other. The dataset comprises 3600 photos for training and 1900 images for testing after being divided into these two categories.

5.1.2 Image Editing

We have chosen to conduct image editing to a subset of 1000 images from the dataset in order to test the resilience of an image classification model. This is done to examine how well the model performs when exposed to different kinds of noise or distortion that may appear in real-world situations. We can simulate the impact of these adverse settings and evaluate how effectively the model is able to categorize pictures under such situations by purposefully blurring a portion of the dataset's images.

To achieve this, we opted to use PhotoScape, a photo-processing program that offers a variety of editing options and tools. These tools include fundamental editing functions like crop and resize and more advanced features like blur. Using the blur tool, we may purposefully reduce the image quality and add noise to the dataset, allowing us to spot any flaws or potential areas for development in the model's classification capabilities.

The intentional blurring of images can be a valuable technique in testing the robustness of an image classification model. By exposing the model to different levels of noise and distortion, we can ensure that it can accurately classify images under various conditions. This can

ultimately lead to a more reliable and accurate model overall, as it is better equipped to handle real-world scenarios and accurately classify images that may be degraded or difficult to interpret.

5.1.3 AI Models

The three models, VGG19, ResNet50, and CNN, were selected to be compared with one another to determine which had the best performance. It is chosen according to the overall accuracy achieved. Initially, ResNet50 was chosen, as it has more convolution layers, becoming the preferred model for diabetic retinopathy detection and other medical image analysis tasks. Additionally, models VGG-19 and CNN were chosen because they are proven deep learning architectures that can achieve good accuracy in image classification tasks, and they can be fine-tuned for the specific task of diabetic retinopathy detection.

5.1.1.3 Building the Algorithm

We have split our dataset into training, validation, and testing. This is done to avoid overfitting the model to the training data while still tuning the model's hyperparameters and evaluating its performance on new data.

1. Training set: This is the portion of the dataset on which the machine learning model is trained. The model is trained on this set by modifying its parameters to reduce the discrepancy between the output on this set's expected and actual outputs. We have trained the model on 80% of the dataset in our implementation.
2. Validation Set: When the model has been trained on the training set, its performance is assessed and its hyperparameters are adjusted on the validation set. A method to gauge how effectively the model generalizes to new, untested data is through the validation set. Hence, we have chosen 20% of the dataset to be validated.
3. Testing Set: The model's performance is evaluated using the testing set after it has been trained and optimized using the training and validation sets. The testing set yields a fair assessment of how well the model performs on new images that it has yet to see. 20% of the dataset was created for testing.

The reason for using 80% training and 20% testing split is to evaluate how well the model can generalize to new, unseen data. When a machine learning model is trained on a dataset, it can memorize the training data instead of recognizing general patterns. To evaluate the model's ability to generalize, it is important to test it on a separate set of data that was not used in the training process. The commonly used 80-20 split provides enough training data for the model to learn from while still leaving a substantial amount of data for testing to measure the model's performance.

In order to enhance the performance of our algorithms, we have done several configurations while building the algorithms:

1. Data reading and configurations: First, the data was read using the Pandas library to read the CSV file containing the image filenames and corresponding labels. Then, we performed some data augmentation and image pre-processing. The images of size 224x224 were converted from BRG to RGB, the default color space used by OpenCV. Weights have been added to blend the input image with a blurred version to enhance the details in the image while reducing unwanted noise. After image pre-processing, the splitting of the dataset was done.
2. Data augmentation: Data augmentation is done to increase the diversity of training the data to improve the model's ability to generalize to new images. Images have been rescaled to achieve the pixel values between 0 and 1, rotated by 20 degrees, shifted, and horizontally flipped.
3. CNN layering: We have used a pre-trained model for the VGG19 and ResNet-50 models. A global average pooling layer is added to the model. Then, a fully connected layer with 256 neurons and a ReLU activation. And finally, a fully connected output layer with 2 neurons and a sigmoid activation function to predict the probability of each class.
4. Training the model: We have set the number of epochs to 10. The size of the batch, which is the number of samples that will be processed through the network at once, is 64. The pre-trained weights were from the ImageNet dataset.
5. Compiling the model: To configure the learning process of the model, an optimizer and a loss function are employed. We have used the Adam optimizer and a binary cross-entropy for the loss function to be used during training.

In the following section, we will be discussing the training and evaluation of multiple AI models.

5.1.1.4 Training AI model

The first training was done using the APTOS dataset. Thus, the models were trained and tested on a collection of retinal images with various degrees of diabetic retinopathy. The performance is measured with various performance measures, including Precision, Sensitivity, and F1-Score. Also, performance graphs, including training and validation accuracy, loss curves, and confusion matrix, are plotted for each architecture trained for 10 epochs using the Adam Optimizer.

- Performance Evaluation:

We used different metrics to evaluate our model's accuracy. The effectiveness of machine learning models is assessed using evaluation metrics.

A confusion matrix is an $N \times N$ matrix, where N is the number of predicted classes. There are three classes for the problem in hand; hence we get a 3x3 matrix. The confusion matrices represent counts from the following values:

- TN: True Negative shows the number of negative examples classified accurately.
- TP: True Positive indicates the number of positive examples classified accurately.
- FP: False Positive value shows the number of actual negative examples categorized as positive.
- FN: False The number of positive instances that are labeled as negative.

The most frequently used performance metrics for image classification are accuracy, precision, sensitivity, specificity, F-score values, and Kappa score.

- Accuracy: Accuracy in a classification problem is the proportion of the correct number of predictions.
$$\text{Accuracy} = (TP + TN) / (TP + FP + FN + TN)$$
- Precision: Precision calculates the correct prediction rate divided by the model's total optimistic prediction.
$$\text{Precision} = TP / (TP + FP)$$
- Sensitivity: Sensitivity is calculated as the ratio of correctly predicted positive values divided by the total number of positive values.
$$\text{Sensitivity} = TP / (TP + FN)$$
- Specificity: Specificity is calculated as the ratio of correctly pessimistic predictions divided by the total number of negative values.
$$\text{Specificity} = (TN) / (TN + FP)$$
- F1-Score: A harmonic average of the accuracy and recall values for a classification task is known as the F1-Score.
$$\text{F1-Score} = 2 * [(Precision * Recall) / (Precision + Recall)]$$
- Kappa: The kappa coefficient measures the agreement between observed and predicted accuracies.
$$\text{Kappa} = (\text{observed accuracy} - \text{predicted accuracy}) / (1 - \text{predicted accuracy})$$

5.2 Experimental Results and Discussions

The table below shows the different experiments we have done. Each model has undergone some experiments, and as a result, the training and testing datasets for each experiment have been described.

Table 2. Experiments done in our project.

Experiment no.	Objective	Training set	Testing set
1	- Comparing between our modified ML models and the pre-trained models	APTOS dataset.	Testing using images of a normal eye and a diabetic eye.
2	- Testing different ML models on the APTOS dataset.	APTOS dataset.	Testing using images of a normal eye and a diabetic eye.
3	- Testing the ability of the model to predict correctly with images it was not trained on.	APTOS dataset with extra distorted images.	Testing using images of a normal eye and a diabetic eye.

Experiment 1:

The first experiment was done to compare between models that are pre-trained on the ImageNet dataset and our own models that we have modified using the pre-trained models. For our modified models, we have used pre-trained models and chose to freeze the first 100 layers. This is done due to the fact that the lower layers of pre-trained models usually learn low-level features, and we want to preserve these weights and only fine-tune the layers for our specific task. We also added an extra output layer with a global average pooling layer to reduce spatial dimensions of the features and dense layer with a sigmoid activation function to classify into two classes.

Table 3. Comparison between the models before and after modification

Models	VGG-19	ResNet-50
Training accuracy before modification	90%	92%
Training accuracy after modification	93%	95%

Comparing between the two models, VGG-19 and ResNet-50, the accuracy of our own modified models for both models have higher accuracies. Hence, we will be completing our next experiments using our own modified models.

Experiment 2:

The second experiment was done using the APTOS dataset. Accordingly, the models were trained and tested on a collection of retinal images with various degrees of diabetic retinopathy. The table below shows the evaluation of a single model and its testing scores.

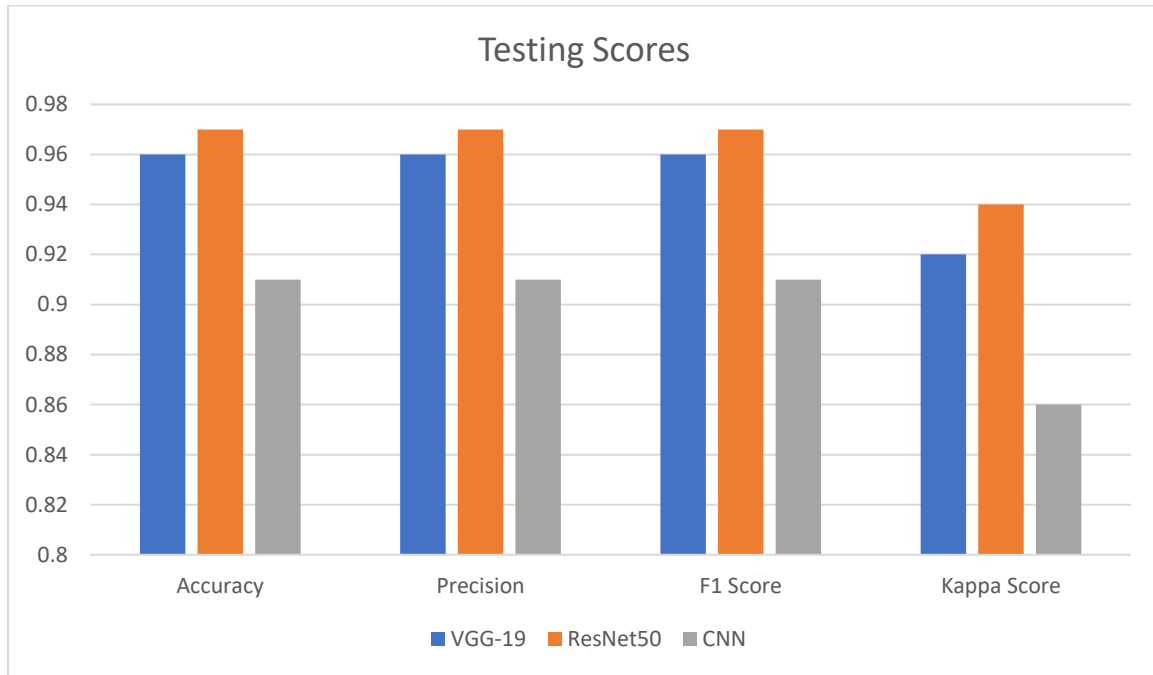


Figure 18 Graphical representation of scores on the APTOS Dataset

Two other metrics are used to evaluate and compare the models:

- Training time per epoch: The time it takes to complete one full iteration of the training process for a model.
- Memory usage: How much memory is used during the training process.

Table 4. Measures for VGG-19, ResNet-50 and CNN.

Measures	VGG-19	ResNet-50	CNN
Training accuracy	93%	95%	92%
Training time per epoch	2000s	1500s	550s
Memory usage	8GB	4.24GB	3GB

We evaluated the models using accuracy, precision, F1, and Kappa scores. Comparing the three different models, we found that the ResNet50 best distinguishes between the classes

(Figure 18). It got a training accuracy of 95% and took 30 minutes on a single epoch. Hence, we select the ResNet50 model as the most suitable model for our system.

- ResNet50 Architecture performance

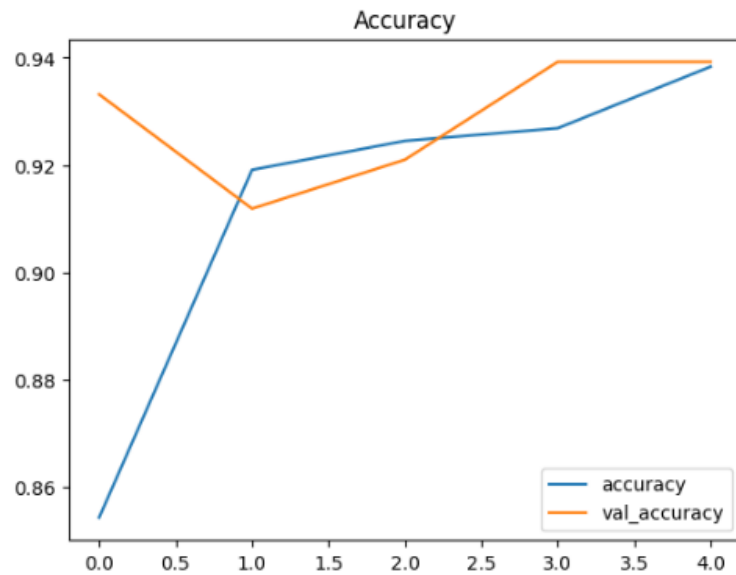


Figure 19. Training and Validation Accuracy Curve – ResNet50

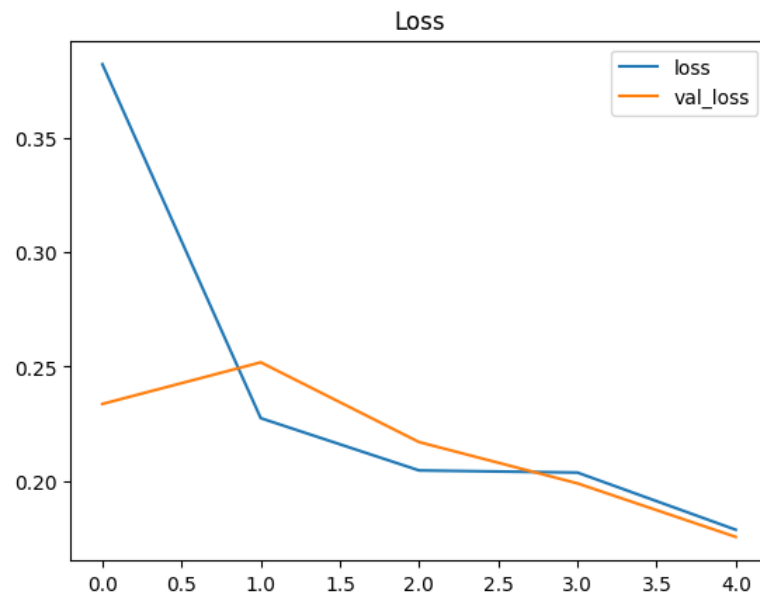


Figure 20. Training and Validation Loss Curve - ResNet50

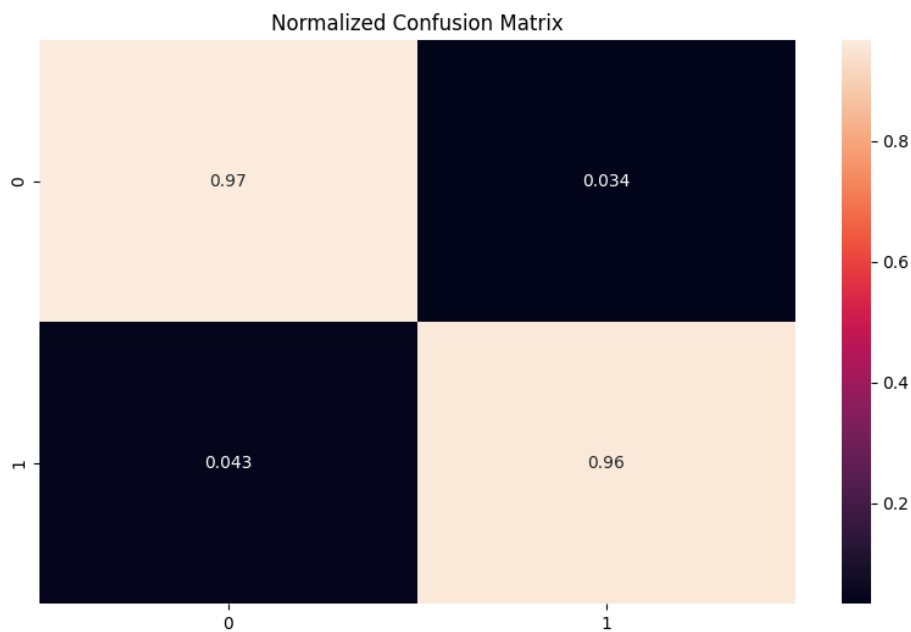


Figure 21. Confusion Matrix – ResNet50

Experiment 3:

- Results: The second experiment was done to evaluate the performance of each model with additional distorted images added to the dataset. Figure 20 shows images from the dataset after adding blurred images.

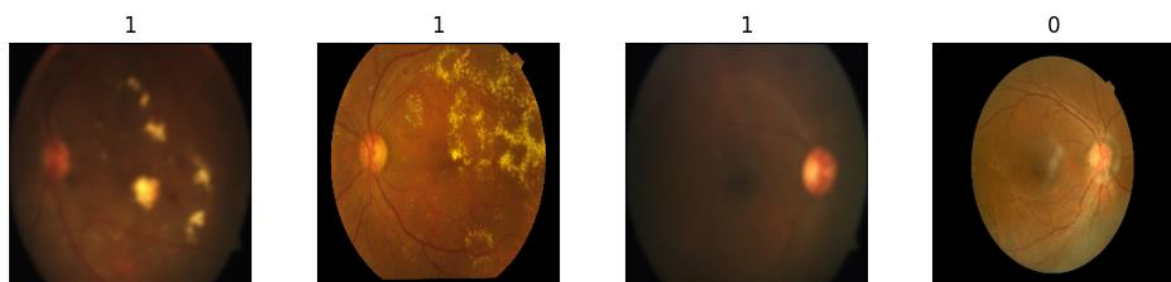


Figure 22. Images from the dataset after distortion

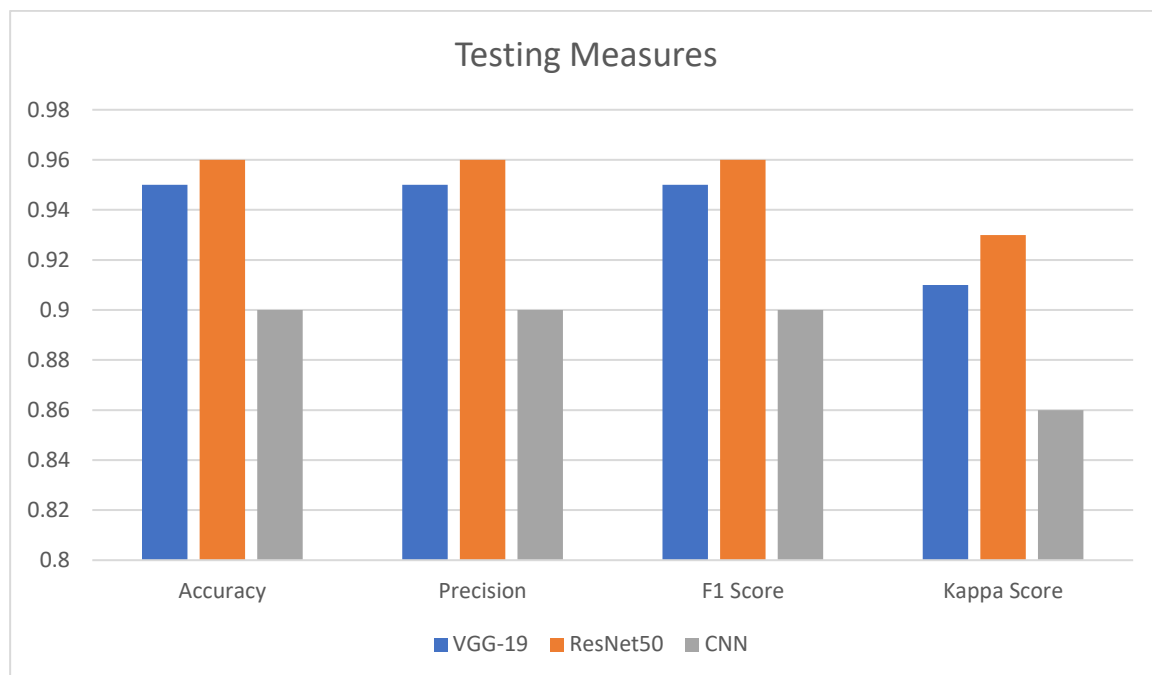


Figure 23. Graphical representation of scores on the APTOS Dataset

As dissipated in Figure 21, with the addition of 1000 distorted images, each model's performance has decreased by 1%. The ResNet50 model has achieved higher accuracy and kappa score than the VGG16 and CNN models. The CNN model was the fastest to train but had the lowest accuracy and kappa score. Hence, we concluded that ResNet50 would be the most efficient model with the best results overall.

6 List of Constraints

This chapter will cover limits and constraints, including environmental, ethical, social, and economic issues.

- Patients can use the system without any assistance from a trained clinician.
- The proposed Design may need to be faster to respond during the first few trials because the software model needs time to recognize the person's retina image.
- Spots or streaks indicate a dirty camera lens. Consequently, camera hygiene may impact the results.
- The image may be compromised or distorted if the patient has corneal edema, cataracts, or other media opacities.

6.1 Environmental Concerns

The design of our project has not identified any major environmental concerns; as a result, none have been addressed.

6.2 Ethical and Social Issues

To ensure the safety and comfort of our targeted consumers, we must comply with the code of ethics and safety regulations, as we are targeting all segments of society, including those with diabetes. In addition, the publication of any consumer-related information, such as the use of retinal fundus images collected for other reasons without the consumer's consent, is considered unethical.

6.3 Economic Issues

It can be expensive to develop a diabetic retinopathy detection system since it needs specialist tools like retinal cameras and imaging software. This may be a barrier to admission for healthcare providers or consumers who want to use such a system. However, by concentrating on the system's software component, our team aims to save expenses. We desire to improve the accessibility and affordability of diabetic retinopathy detection by creating algorithms that can evaluate retinal images acquired by a standard camera.

As technology advances, our system will also be able to change and adapt to the evolving state of healthcare and medical technology. It will be advantageous for both patients and healthcare providers to invest in creating and sustaining such a system since this will ultimately increase revenue and enhance patient outcomes.

7 List of Specifications

7.1 Parts List and Estimated Budget

No hardware parts were used to implement our project. Our Design focused on building an AI model using Python, TensorFlow, Keras, and OpenCV. Python is one of the most used programming languages in order to build AI and deep learning models. These models include CNN models. TensorFlow is an open-source software library for building deep learning models. Keras is a high-level API that sits on top of TensorFlow, making it easier to build and train models. Open-source computer vision (OpenCV) is a library that provides a variety of image processing algorithms and other tools. The estimated budget and parts list to build this model that can detect diabetic retinopathy using CNN is:

Python, TensorFlow, Keras, OpenCV: Free

8 Difficulties Encountered

The first challenge that we faced at the start of implementing our project was finding the most suitable dataset that was large enough with diverse images. Since diabetic retinopathy is a relatively rare disease, there were some imbalances in the dataset with a large number of normal images and fewer images with diabetic retinopathy. This led us to create our own binary dataset, 0 for no diabetic retinopathy and 1 for all diabetic retinopathy stages combined.

The selection of the appropriate architecture and hyperparameters presented our second obstacle. Three distinct models were trained using the same pre-processing methods. Finding the most suitable set of pre-processing techniques to use on the images in order to achieve the best accuracy was challenging.

Finally, the last significant factor that affected us was time. Training a model may take a while, depending on the dataset's size and the architecture's complexity. Having to repeat the training for each model until we reached the maximum accuracy was a challenge regarding time.

Conclusion

Our senior design project aimed to develop an efficient and accurate diabetic retinopathy detection system by leveraging the APTOS dataset containing high-resolution retina images. We employed deep learning techniques, specifically convolutional neural networks (CNNs) like ResNet and VGG, for image classification to accomplish this. The primary objective of our system was to detect and classify diabetic retinopathy in retinal images, thereby aiding medical professionals in making informed decisions and providing timely treatment to patients.

To ensure the optimal performance of our CNNs, we pre-processed and augmented the images before feeding them into our models. Several performance criteria, including accuracy, precision, and F1 score, were used to assess the efficiency of our system. By reducing human errors and the time consumed in diagnosing diabetic retinopathy, our system aimed to improve the overall efficiency and accuracy of the diagnosis process.

We believe that our project has significant potential to positively impact diabetic patients, especially those in remote or underserved areas with limited access to medical facilities or expertise. The diabetic retinopathy detection system we developed could aid in early detection and prompt treatment, potentially preventing long-term vision loss and other complications associated with the disease.

9 References

- [1] Y. Zheng, M. He, and N. Congdon, "The worldwide epidemic of diabetic retinopathy," *Indian J Ophthalmol*, vol. 60, no. 5, p. 428, 2012, doi: 10.4103/0301-4738.100542.
- [2] S. Wild, G. Roglic, A. Green, R. Sicree, and H. King, "Global Prevalence of Diabetes," *Diabetes Care*, vol. 27, no. 5, pp. 1047–1053, May 2004, doi: 10.2337/diacare.27.5.1047.
- [3] M. Bhaskaranand *et al.*, "Automated Diabetic Retinopathy Screening and Monitoring Using Retinal Fundus Image Analysis," *J Diabetes Sci Technol*, vol. 10, no. 2, pp. 254–261, Mar. 2016, doi: 10.1177/1932296816628546.
- [4] E. Y. K. Ng, J. H. Tan, U. R. Acharya, and J. S. Suri, Eds., *Human Eye Imaging and Modeling*. CRC Press, 2012. doi: 10.1201/b12081.
- [5] N. Salamat, M. M. S. Missen, and A. Rashid, "Diabetic retinopathy techniques in retinal images: A review," *Artif Intell Med*, vol. 97, pp. 168–188, Jun. 2019, doi: 10.1016/j.artmed.2018.10.009.
- [6] M. R. K. Mookiah, U. R. Acharya, C. K. Chua, C. M. Lim, E. Y. K. Ng, and A. Laude, "Computer-aided diagnosis of diabetic retinopathy: A review," *Comput Biol Med*, vol. 43, no. 12, pp. 2136–2155, Dec. 2013, doi: 10.1016/j.compbiomed.2013.10.007.
- [7] J. Krause *et al.*, "Grader variability and the importance of reference standards for evaluating machine learning models for diabetic retinopathy," Oct. 2017, doi: 10.1016/j.ophtha.2018.01.034.
- [8] D. Nagpal, S. N. Panda, M. Malarvel, P. A. Pattanaik, and M. Zubair Khan, "A review of diabetic retinopathy: Datasets, approaches, evaluation metrics and future trends," *Journal of King Saud University - Computer and Information Sciences*, vol. 34, no. 9, pp. 7138–7152, Oct. 2022, doi: 10.1016/j.jksuci.2021.06.006.
- [9] M. M. Abdelsalam, "Effective blood vessels reconstruction methodology for early detection and classification of diabetic retinopathy using OCTA images by artificial neural network," *Inform Med Unlocked*, vol. 20, p. 100390, 2020.

- [10] J. Sahlsten *et al.*, “Deep Learning Fundus Image Analysis for Diabetic Retinopathy and Macular Edema Grading,” *Sci Rep*, vol. 9, no. 1, p. 10750, 2019, doi: 10.1038/s41598-019-47181-w.
- [11] G. G. Gardner, D. Keating, T. H. Williamson, and A. T. Elliott, “Automatic detection of diabetic retinopathy using an artificial neural network: a screening tool,” *British Journal of Ophthalmology*, vol. 80, no. 11, p. 940, Nov. 1996, doi: 10.1136/bjo.80.11.940.
- [12] L. Dai *et al.*, “A deep learning system for detecting diabetic retinopathy across the disease spectrum,” *Nat Commun*, vol. 12, no. 1, p. 3242, 2021, doi: 10.1038/s41467-021-23458-5.
- [13] X. Dong, S. Du, W. Zheng, C. Cai, H. Liu, and J. Zou, “Evaluation of an Artificial Intelligence System for the Detection of Diabetic Retinopathy in Chinese Community Healthcare Centers,” *Front Med (Lausanne)*, vol. 9, 2022, [Online]. Available: <https://www.frontiersin.org/articles/10.3389/fmed.2022.883462>
- [14] P. Ruamviboonsuk *et al.*, “Real-time diabetic retinopathy screening by deep learning in a multisite national screening programme: a prospective interventional cohort study,” *Lancet Digit Health*, vol. 4, no. 4, pp. e235–e244, Apr. 2022, doi: 10.1016/S2589-7500(22)00017-6.
- [15] J.-H. Wu, T. Y. A. Liu, W.-T. Hsu, J. H.-C. Ho, and C.-C. Lee, “Performance and Limitation of Machine Learning Algorithms for Diabetic Retinopathy Screening: Meta-analysis,” *J Med Internet Res*, vol. 23, no. 7, p. e23863, Jul. 2021, doi: 10.2196/23863.
- [16] R. N. Maamari, J. D. Keenan, D. A. Fletcher, and T. P. Margolis, “A mobile phone-based retinal camera for portable wide field imaging,” *British Journal of Ophthalmology*, vol. 98, no. 4, pp. 438–441, 2014, doi: 10.1136/bjophthalmol-2013-303797.
- [17] G. Lim, V. Bellemo, Y. Xie, X. Q. Lee, M. Y. T. Yip, and D. S. W. Ting, “Different fundus imaging modalities and technical factors in AI screening for diabetic retinopathy: a review,” *Eye and Vision*, vol. 7, no. 1, pp. 1–13, Dec. 2020, doi: 10.1186/S40662-020-00182-7/FIGURES/4.
- [18] A. Grzybowski and P. Brona, “Analysis and comparison of two artificial intelligence diabetic retinopathy screening algorithms in

- a pilot study: Idx-dr and retinalyze,” *J Clin Med*, vol. 10, no. 11, Jun. 2021, doi: 10.3390/jcm10112352.
- [19] M. Bhaskaranand *et al.*, “The value of automated diabetic retinopathy screening with the EyeArt system: A study of more than 100,000 consecutive encounters from people with diabetes,” *Diabetes Technol Ther*, vol. 21, no. 11, pp. 635–643, Nov. 2019, doi: 10.1089/dia.2019.0164.
- [20] P. Bawankar *et al.*, “Sensitivity and specificity of automated analysis of single-field non-mydriatic fundus photographs by Bosch DR Algorithm—Comparison with mydriatic fundus photography (ETDRS) for screening in undiagnosed diabetic retinopathy,” *PLoS One*, vol. 12, no. 12, Dec. 2017, doi: 10.1371/journal.pone.0189854.
- [21] “Artificial Intelligence | Fastweb.”
<https://www.fastweb.com/directory/scholarships-for-artificial-intelligence-majors> (accessed Nov. 14, 2022).
- [22] “Types of Machine Learning - Javatpoint.”
<https://www.javatpoint.com/types-of-machine-learning> (accessed Nov. 14, 2022).
- [23] “Artificial Intelligence and Machine Learning for Digital Pathology: State-of-the-Art and Future Challenges [1st ed.] 9783030504014, 9783030504021 - DOKUMEN.PUB.”
<https://dokumen.pub/artificial-intelligence-and-machine-learning-for-digital-pathology-state-of-the-art-and-future-challenges-1st-ed-9783030504014-9783030504021.html> (accessed Nov. 24, 2022).
- [24] “What is Deep Learning? | IBM.”
<https://www.ibm.com/cloud/learn/deep-learning> (accessed Nov. 14, 2022).
- [25] “McCulloch-Pitts Neuron — Mankind’s First Mathematical Model Of A Biological Neuron | by Akshay L Chandra | Towards Data Science.” <https://towardsdatascience.com/mcculloch-pitts-model-5fdf65ac5dd1> (accessed Nov. 14, 2022).
- [26] “Computer Vision.”
<https://cs.nyu.edu/~fergus/teaching/vision/index.html> (accessed Oct. 24, 2022).

- [27] “Artificial Neural Network | Beginners Guide to ANN.”
<https://www.analyticsvidhya.com/blog/2021/05/beginners-guide-to-artificial-neural-network/> (accessed Nov. 24, 2022).
- [28] “Understanding Backpropagation Algorithm | by Simeon Kostadinov | Towards Data Science.”
<https://towardsdatascience.com/understanding-backpropagation-algorithm-7bb3aa2f95fd> (accessed Nov. 24, 2022).
- [29] “Introduction Multilayer Perceptron Neural Networks | DTREG.”
<https://www.dtreg.com/solution/multilayer-perceptron-neural-networks> (accessed Oct. 24, 2022).
- [30] “Support Vector Machine — Introduction to Machine Learning Algorithms | by Rohith Gandhi | Towards Data Science.”
<https://towardsdatascience.com/support-vector-machine-introduction-to-machine-learning-algorithms-934a444fca47> (accessed Oct. 24, 2022).
- [31] “Support Vector Machine(SVM): A Complete guide for beginners.”
<https://www.analyticsvidhya.com/blog/2021/10/support-vector-machinessvm-a-complete-guide-for-beginners/> (accessed Nov. 24, 2022).
- [32] “Support Vector Machine (SVM) Algorithm - Javatpoint.”
<https://www.javatpoint.com/machine-learning-support-vector-machine-algorithm> (accessed Nov. 14, 2022).
- [33] “Advantages of Support Vector Machines (SVM).”
<https://iq.opengenus.org/advantages-of-svm/> (accessed Oct. 24, 2022).
- [34] “Random Forest | Introduction to Random Forest Algorithm.”
<https://www.analyticsvidhya.com/blog/2021/06/understanding-random-forest/> (accessed Oct. 24, 2022).
- [35] “Random forest Algorithm in Machine learning | Great Learning.” <https://www.mygreatlearning.com/blog/random-forest-algorithm/> (accessed Oct. 24, 2022).
- [36] “Convolutional neural network - Wikipedia.”
<https://en.wikipedia.org/wiki?curid=40409788> (accessed Nov. 24, 2022).
- [37] “Convolutional Neural Network Architecture | CNN Architecture.”

- <https://www.analyticsvidhya.com/blog/2020/10/what-is-the-convolutional-neural-network-architecture/> (accessed Nov. 13, 2022).
- [38] “ImageNet.” <https://www.image-net.org/update-mar-11-2021.php> (accessed Nov. 13, 2022).
- [39] J. Jaworek-Korjakowska, P. Kleczek, and M. Gorgon, “Melanoma thickness prediction based on convolutional neural network with VGG-19 model transfer learning,” *IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops*, vol. 2019-June, pp. 2748–2756, Jun. 2019, doi: 10.1109/CVPRW.2019.00333.
- [40] “Architecture of DenseNet-121.” <https://iq.opengenus.org/architecture-of-densenet121/> (accessed Nov. 13, 2022).
- [41] “densenet121_spXhNmT.png (678×516).” https://production-media.paperswithcode.com/models/densenet121_spXhNmT.png (accessed Nov. 13, 2022).
- [42] R. Indraswari, R. Rokhana, and W. Herulambang, “Melanoma image classification based on MobileNetV2 network,” in *Procedia Computer Science*, Elsevier B.V., 2021, pp. 198–207. doi: 10.1016/j.procs.2021.12.132.
- [43] “Diabetic Retinopathy Detection | Kaggle.” <https://www.kaggle.com/c/diabetic-retinopathy-detection/data> (accessed Nov. 14, 2022).
- [44] “GitHub - dimitreOliveira/APTOS2019BlindnessDetection: :3rd_place_medal: (Bronze medal - 163rd place - Top 6%) Repository for the ‘APTOS 2019 Blindness Detection’ Kaggle competition.” <https://github.com/dimitreOliveira/APTOS2019BlindnessDetection> (accessed Nov. 14, 2022).
- [45] B. Sumathy *et al.*, “Prediction of Diabetic Retinopathy Using Health Records With Machine Learning Classifiers and Data Science,” *International Journal of Reliable and Quality E-Healthcare*, vol. 11, no. 2, pp. 1–16, Apr. 2022, doi: 10.4018/IJRQEH.299959.

Appendices

Appendix 1 – ResNet50 Python code

```
# -*- coding: utf-8 -*-
"""RESNET50-NO DISTORTION

Automatically generated by Colaboratory.

Original file is located at
https://colab.research.google.com/drive/1a-7QA0NqoAAOpGvkg6pmI-VS0AheHyXB
"""

import numpy as np
import pandas as pd
from pathlib import Path
import os.path
import matplotlib.pyplot as plt
from IPython.display import Image, display, Markdown
import matplotlib.cm as cm
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
import tensorflow as tf
from time import perf_counter
import seaborn as sns
from tensorflow import keras
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from sklearn.model_selection import train_test_split
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.applications.resnet50 import ResNet50

def printmd(string):
    # Print with Markdowns
    display(Markdown(string))

from google.colab import drive
drive.mount('/content/drive')

image_dir = Path('/content/drive/MyDrive/01/train')

#To get filepaths
```

```
filepaths = list(image_dir.glob(r'**/*.png'))
labels = list(map(lambda x:
os.path.split(os.path.split(x)[0])[1], filepaths))

filepaths = pd.Series(filepaths, name='Filepath').astype(str)
labels = pd.Series(labels, name='Label')

# Concatenate filepaths and labels
image_df = pd.concat([filepaths, labels], axis=1)

# Shuffle the DataFrame and reset index
image_df = image_df.sample(frac=1).reset_index(drop = True)

# Show the result
image_df.head(3)

"""# Visualization"""

# Display some pictures of the dataset with their labels
fig, axes = plt.subplots(nrows=3, ncols=4, figsize=(10, 7),
                        subplot_kw={'xticks': [], 'yticks':
[]})

for i, ax in enumerate(axes.flat):
    ax.imshow(plt.imread(image_df.Filepath[i]))
    ax.set_title(image_df.Label[i])
plt.tight_layout()
plt.show()

def preprocess_image(image_path, desired_size=300):
    im = cv2.imread(image_path)
    im = cv2.cvtColor(im, cv2.COLOR_BGR2RGB)
    im = cv2.resize(im, (desired_size, desired_size))
    im = cv2.addWeighted(im, 4, cv2.blur(im, ksize=(10,10)), -
4, 128)
    return im

# Display the number of pictures of each category
vc = image_df['Label'].value_counts()
plt.figure(figsize=(9,5))
sns.barplot(x = vc.index, y = vc, palette = "rocket")
plt.title("Number of pictures of each category", fontsize =
15)
plt.show()

"""# Load the Images with a generator<a class="anchor"
id="2"></a>"""
```

```
def create_gen():
    # Load the Images with a generator and Data Augmentation
    train_generator =
tf.keras.preprocessing.image.ImageDataGenerator(

preprocessing_function=tf.keras.applications.mobilenet_v2.prep
rocess_input,
    validation_split=0.1
)

    test_generator =
tf.keras.preprocessing.image.ImageDataGenerator(

preprocessing_function=tf.keras.applications.mobilenet_v2.prep
rocess_input
)

    train_images = train_generator.flow_from_dataframe(
        dataframe=train df,
        x_col='Filepath',
        y_col='Label',
        target_size=(224, 224),
        color_mode='rgb',
        class_mode='categorical',
        batch_size=64,
        shuffle=True,
        seed=0,
        subset='training',
        rotation_range=30, # Uncomment to use data
augmentation
        zoom_range=0.15,
        width_shift_range=0.2,
        height_shift_range=0.2,
        shear_range=0.15,
        horizontal_flip=True,
        fill_mode="nearest"
    )

    val_images = train_generator.flow_from_dataframe(
        dataframe=train df,
        x_col='Filepath',
        y_col='Label',
        target_size=(224, 224),
        color_mode='rgb',
        class_mode='categorical',
        batch_size=64,
```

```
        shuffle=True,
        seed=0,
        subset='validation',
        rotation_range=30,
        zoom_range=0.15,
        width_shift_range=0.2,
        height_shift_range=0.2,
        shear_range=0.15,
        horizontal_flip=True,
        fill_mode="nearest"
    )

    test_images = test_generator.flow_from_dataframe(
        dataframe=test_df,
        x_col='Filepath',
        y_col='Label',
        target_size=(224, 224),
        color_mode='rgb',
        class_mode='categorical',
        batch_size=64,
        shuffle=False
    )

    return
train_generator, test_generator, train_images, val_images, test_images

# Separate in train and test data
train_df, test_df = train_test_split(image_df, train_size=0.9,
shuffle=True, random_state=1)

"""# resnet50<a class="anchor" id="4"></a>"""

import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.image import
ImageDataGenerator
from sklearn.model_selection import train_test_split
from tensorflow.keras.optimizers.legacy import Adam

# Define the ResNet50 model
resnet50_model = keras.applications.ResNet50(
    include_top=False,
    weights='imagenet',
```

```
        input_shape=(224, 224, 3)
    )

    # Set the ResNet50 model to be non-trainable
    resnet50_model.trainable = False

    from tensorflow.keras.models import Sequential
    from tensorflow.keras.layers import Dense, Flatten, Conv2D,
    MaxPooling2D
    from tensorflow.keras.applications.resnet50 import ResNet50
    from tensorflow.keras.callbacks import ReduceLROnPlateau,
    EarlyStopping
    from tensorflow.keras import models, layers, optimizers

    # Create the generators
    train_generator, test_generator, train_images, val_images,
    test_images = create_gen()

    # Load the pretained model
    resnet50_model = ResNet50(
        include_top=False,
        weights='imagenet',
        input_shape=(224, 224, 3)
    )

    # Freeze the layers up to a specific layer
    for layer in resnet50_model.layers[:100]:
        layer.trainable = False

    # Add a new output layer
    x = resnet50_model.output
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dense(256, activation='relu')(x)
    predictions = layers.Dense(2, activation='sigmoid')(x)

    # Define the full model
    model = models.Model(inputs=resnet50_model.input,
        outputs=predictions)

    # Fine-tune the model
    model.compile(
        optimizer=optimizers.Adam(learning_rate=0.0001),
        loss='binary_crossentropy',
        metrics=['accuracy', 'AUC']
    )
```

```
# define the callbacks
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2,
patience=3, min_lr=0.0001)
early_stop = EarlyStopping(monitor='val_loss', patience=5)

history = model.fit(
    train_images,
    epochs=10,
    validation_data=val_images,
    callbacks=[reduce_lr, early_stop]
)

pd.DataFrame(history.history)[['accuracy', 'val_accuracy']].plot()
plt.title("Accuracy")
plt.show()

pd.DataFrame(history.history)[['loss', 'val_loss']].plot()
plt.title("Loss")
plt.show()

pd.DataFrame(history.history)[['accuracy', 'val_accuracy']].plot()
plt.title("Accuracy")
plt.show()

pd.DataFrame(history.history)[['loss', 'val_loss']].plot()
plt.title("Loss")
plt.show()

results = model.evaluate(test_images, verbose=0)

printmd(" ## Test Loss: {:.5f}".format(results[0]))
printmd("## Accuracy on the test set:
{:.2f}%".format(results[1] * 100))
print('\n')

# Predict the label of the test_images
pred = model.predict(test_images)
pred = np.argmax(pred, axis=1)

# Map the label
labels = (train_images.class_indices)
labels = dict((v,k) for k,v in labels.items())
pred = [labels[k] for k in pred]
```

```
from sklearn.metrics import classification_report
y_test = list(test_df.Label)
from sklearn import metrics
print('Accuracy:',
      np.round(metrics.accuracy_score(y_test,pred),5))
print('Precision:',
      np.round(metrics.precision_score(y_test,pred,
      average='weighted'),2))
print('Recall:', np.round(metrics.recall_score(y_test,pred,
      average='weighted'),2))
print('F1 Score:', np.round(metrics.f1_score(y_test,pred,
      average='weighted'),2))
print('Cohen Kappa Score:',
      np.round(metrics.cohen_kappa_score(y_test,pred),2))
print(classification_report(y_test, pred))

cf_matrix = confusion_matrix(y_test, pred, normalize='true')
plt.figure(figsize = (10,6))
sns.heatmap(cf_matrix, annot=True, xticklabels =
sorted(set(y_test)), yticklabels = sorted(set(y_test)))
plt.title('Normalized Confusion Matrix')
plt.show()

import psutil

# Get the system memory usage
mem = psutil.virtual_memory()

# Print the memory usage in GB
print(f"Total memory: {mem.total/1e9:.2f} GB")
print(f"Available memory: {mem.available/1e9:.2f} GB")
print(f"Used memory: {mem.used/1e9:.2f} GB")

test_data =
pd.read_csv("/content/drive/MyDrive/dataset/test.csv")
test_data["filename"] = test_data["id_code"].map(lambda
x:x+".png")
test_data.head()

filenames=test_generator.filenames
results=pd.DataFrame({"id_code":filenames,
                      "diagnosis":np.argmax(predictions,axis=
1)})
results['id_code'] = results['id_code'].map(lambda x: str(x)
[:-4])
results.to_csv("submission.csv",index=False)

test_dir = '/content/drive/MyDrive/01/test/test_images'
test_csv = '/content/drive/MyDrive/01/test/test.csv'
test_df = pd.read_csv(test_csv)
test_df['id_code'] = test_df['id_code'] + ".png"
test_df['diagnosis'] = np.zeros(test_df.shape[0])
img_size = (224, 224)
```

```
img_size = (224, 224)
ts_length = len(test_df)
test_batch_size = max(sorted([ts_length // n for n in range(1,
ts_length + 1) if ts_length%n == 0 and ts_length/n <= 80]))

def scalar(img):
    return img

tst_gen = ImageDataGenerator(preprocessing_function= scalar)
test_gen = tst_gen.flow_from_dataframe( test_df, x_col=
'id_code', y_col= 'diagnosis', directory= test_dir,
target_size= img_size,
                                color_mode= 'rgb',
class_mode='raw', shuffle= False, batch_size= test_batch_size)

#Get Prediction

preds = model.predict(test_gen)
y_pred = np.argmax(preds, axis=1)
y_pred

test_df['diagnosis'] = y_pred
test_df

import cv2
from google.colab.patches import cv2_imshow

img =
cv2.imread("/content/drive/MyDrive/01/test/test_images/0005cfc
8afb6.png", cv2.IMREAD_COLOR)

cv2_imshow(img)
cv2.waitKey(0)
cv2.destroyAllWindows()

test_df.to_csv('./submission.csv', index=False)
```
